Web and Data Engineering

Vorlesung 06: Data Engineering

Prof. Dr. Michael Granitzer

25 März 2026

Data Engineering

Die Vorlesung verknüpft datengetriebene Web-Systeme mit Pipelines, Speicherarchitekturen und Betriebsfragen rund um Qualität und Skalierung.

Fokus

- wie aus Rohdaten verarbeitbare Datenprodukte werden
- wie Data Warehouse, Data Lake und Lakehouse sich unterscheiden — und warum columnar Formate (Parquet, Arrow) der gemeinsame Nenner sind
- wie das MapReduce-Paradigma entstand und sich über Spark, Flink und dbt zu modernen Verarbeitungs-Stacks weiterentwickelt hat
- warum Batch- und Stream-Verarbeitung unterschiedliche Zuschnitte brauchen und wie Lambda- und Kappa-Architektur die Integration lösen
- wie Datenqualität, Governance und Betrieb zusammenwirken

Modul 1

Datenpipelines und Datenprodukte

Leitfrage: Was ist Big Data — und warum überfordert es klassische Systeme?

Welche Daten sammeln wir überhaupt?

Moderne Unternehmen und Plattformen erzeugen Daten aus sehr unterschiedlichen Quellen:

Operative Systeme

- **Case Management:** Audit-Trails, Compliance-Daten
- **CRM / ERP:** SAP, Oracle — Kundendaten, Transaktionen
- **Service Desk:** Tickets, Vorfälle, Reaktionszeiten

Digitale & physische Welt

- **Internet Clickstreams:** Seitenaufrufe, Klicks, Sessions
- **Netflow:** Netzwerkverkehr, Sicherheitsereignisse
- **IoT Device Events:** Sensoren, Maschinen, Fahrzeuge

Jede dieser Quellen hat eigenes Format, eigene Geschwindigkeit, eigene Vertrauenswürdigkeit.

Was macht Daten zu „Big Data"?

Die Fünf Vs von Big Data (IBM)

Big Data ist kein Größenbegriff, sondern ein Eigenschaftsprofil: **Volume, Variety, Velocity, Veracity, Value**.

Die klassische Datenbank-Annahme: Daten passen in Tabellen, kommen in moderatem Tempo, und sind im Wesentlichen korrekt. Alle fünf Vs stellen genau diese Annahmen in Frage.

V1 — Volume: Das Mengenproblem

Datenmengen übersteigen, was einzelne Maschinen verarbeiten können.

Quellen für extreme Volumina

Bereich	Beispiel
Web & E-Commerce	Produktaufrufe, Kaufhistorien, Reviews
Banking	Zahlungstransaktionen weltweit
Social Networks	Posts, Likes, Kommentare
LTE / 5G	Verbindungsprotokolle, Netzwerktelemetrie

Einordnung

> *„Eine einzelne Google-Suche nutzt mehr Rechenleistung als die gesamte Apollo-Raumfahrtmission."*

- Google indiziert ~10 Mrd. Seiten = ca. 200 TB
- Bei 50 MB/s Bandbreite: Lesen allein würde **40 Tage** dauern
- Lösung: Parallelisierung und verteilte Verarbeitung

RDBMS wurden für einzelne, skalierbare Maschinen entworfen. Horizontale Verteilung ist nachträglich schwer einzubauen.

V2 — Variety: Das Formatproblem

Daten kommen nicht mehr nur als Tabellen.

Datenmodell	Beschreibung	Beispiel
Relational	Tabellen, Transaktionen, festes Schema	Bestellungen, Konten
Text	Freitextinhalt, unstrukturiert	Web-Seiten, E-Mails, Tickets
Semi-strukturiert	Schema flexibel, hierarchisch	XML, JSON, Logs
Graph	Knoten und Kanten, Beziehungsnetze	Soziale Netzwerke, RDF
Streaming	Kontinuierlicher Datenstrom, nur einmal lesbar	Sensor-Events, Klicks

Wenn Quellsysteme ihr Format ändern, bricht nachgelagerte Verarbeitung. **Schema-Evolution** ist ein zentrales Problem im Data Engineering.

V3 — Velocity: Das Geschwindigkeitsproblem

Daten erscheinen mit sehr unterschiedlichen Geschwindigkeiten — und Entscheidungen müssen entsprechend zeitnah getroffen werden.

Konkrete Zahlen

Quelle	Geschwindigkeit
Globaler Internettraffic	~50.000 GB/s
Twitter	~6.000 Tweets/Sekunde
Suchanfragen (Google)	~100.000/Sekunde
Kreditkartentransaktionen	tausende/Sekunde weltweit

Konsequenz für Systeme

- Systeme müssen Entscheidungen in **Millisekunden** bis Sekunden treffen
- Betrugserkennung kann nicht auf den nächsten Batch-Job warten
- Alerting auf Netzwerkanomalien erfordert Echtzeit-Verarbeitung

V4 — Veracity: Das Vertrauensproblem

Nicht alle Daten sind verlässlich. Veracity beschreibt den Grad der Unsicherheit.

Quellen von Datenunsicherheit

- Fehlerhafte Eingaben (Tippfehler, falsche Werte)
- Veraltete oder doppelte Datensätze
- Inkonsistente Definitionen zwischen Systemen
- Messfehler bei Sensoren
- Absichtlich verfälschte Daten (Fraud)

Empirische Einordnung

Laut Unternehmensumfragen zweifelt **jeder dritte Entscheidungsträger** an der Qualität der Daten, auf denen Entscheidungen basieren.
Unsichere Daten führen zu unsicheren Modellen — unabhängig von Rechenleistung oder Algorithmus.

Garbage In, Garbage Out: Kein ML-Modell und kein Analysesystem kann systematisch fehlerhafte Eingaben kompensieren.

V5 — Value: Das Nutzenproblem

Daten allein schaffen keinen Wert. Großmengen zu speichern ist kostspielig — der Nutzen muss aktiv erschlossen werden.

Warum klassische Datenbanken scheitern

Klassische RDBMS wurden für eine Welt entworfen, die es so nicht mehr gibt:

Annahme klassischer Systeme	Realität bei Big Data
Daten passen auf eine Maschine	Petabyte-Mengen erfordern Cluster
Schema ist stabil und bekannt	Format ändert sich laufend (Variety)
Schreibvorgänge sind moderat	Millionen Events pro Sekunde (Velocity)
Daten sind im Wesentlichen korrekt	Systematische Qualitätsprobleme (Veracity)
Ausfälle sind Ausnahmen	Bei 1000 Knoten: täglich Ausfälle

Die Fünf Vs erzwingen einen Paradigmenwechsel: von zentralisierten RDBMS zu verteilten, skalierbaren Systemen mit expliziter Qualitätskontrolle.

Mini-Übung: Die 5 Vs im E-Commerce

Ordnen Sie jedem V ein konkretes Beispiel aus einem Online-Shop zu.

V	Ihre Zuordnung
Volume	?
Variety	?
Velocity	?
Veracity	?
Value	?

Musterlösung

V	E-Commerce-Beispiel
Volume	Klick-Logs aller Seitenbesuche (Milliarden Einträge/Tag)
Variety	Produktdaten (strukturiert), Bewertungstexte (Text), Nutzersessions (Semi-strukturiert)
Velocity	Preisanpassungen in Echtzeit bei Flash-Sales
Veracity	Fake-Bewertungen, fehlerhafte Produktattribute, doppelte Kundendatensätze
Value	Empfehlungsalgorithmus steigert durchschnittlichen Warenkorbwert

Big Data ist kein Mengenproblem allein. Die fünf Vs beschreiben ein Eigenschaftsprofil, das klassische Datenbanken systematisch überfordert. Jedes V erfordert andere architektonische Antworten.

Schlüsselbegriffe — Modul 1

Begriff	Erklärung
Big Data	Datensätze, die durch Volume, Variety, Velocity, Veracity oder Value klassische Verarbeitung überfordern
Volume	Datenmenge — von Gigabyte bis Petabyte und darüber hinaus
Variety	Vielfalt der Datenformate und -modelle (relational, Text, Graph, Stream)
Velocity	Geschwindigkeit, mit der Daten erzeugt und verarbeitet werden müssen
Veracity	Verlässlichkeit und Qualität der Daten; Unsicherheit über Korrektheit
Value	Nutzen, der durch Analyse und Anwendung aus Daten gewonnen wird
ETL	Extract, Transform, Load — klassischer Drei-Schritt-Prozess der Datenpipeline
Clickstream	Protokoll der Aktionen eines Nutzers auf einer Website

Modul 2

Speicher- und Verarbeitungsschichten

Leitfrage: Wie gewinnt man Wissen aus Daten?

Data Science — eine Einordnung

Data Science erforscht und wendet Technologien, Systeme und Algorithmen an, um Wissen aus potenziell großen, heterogenen und dynamisch veränderlichen Datensätzen zu gewinnen.

Data Science

Methoden und Systeme zur Wissensgewinnung aus Daten — verbindet Statistik, Informatik und Domänenwissen.

Machine Learning

Algorithmen, die aus vergangener Erfahrung lernen und Entscheidungen treffen, ohne explizit programmiert zu sein.

Artificial Intelligence

Systeme, die intelligentes Verhalten zeigen — ML ist der aktuell dominante Ansatz, AI der übergeordnete Begriff.

Deduktives vs. Induktives Schließen

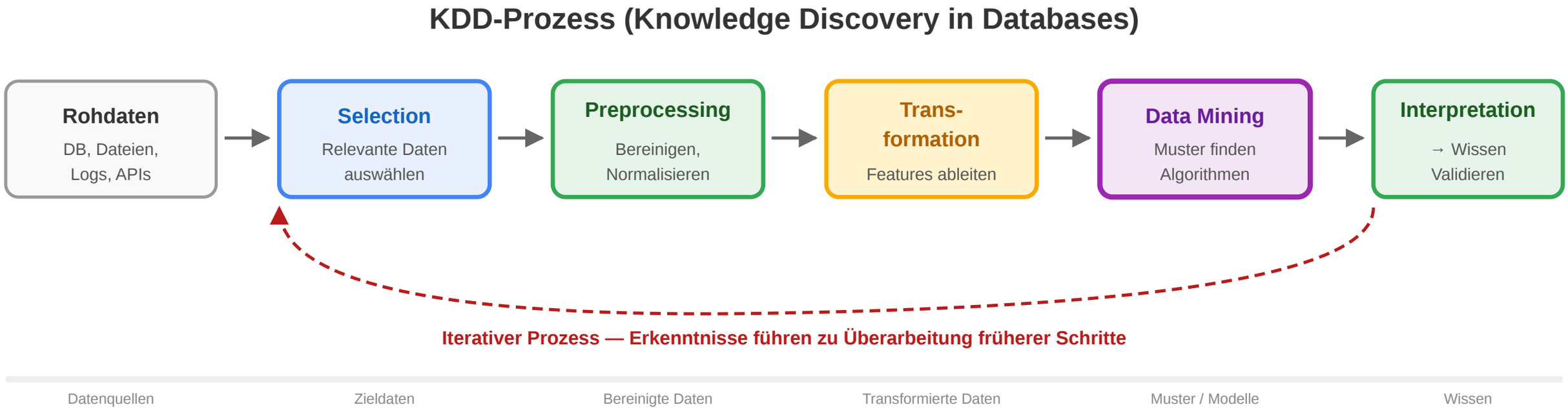
Die Art, wie Wissen gewonnen wird, unterscheidet sich grundlegend:

Aspekt	Deduktives Schließen	Induktives Schließen
Ausgangspunkt	Gegebene Prämissen und Regeln	Beobachtete Datenmuster
Richtung	Allgemein → Spezifisch	Spezifisch → Allgemein
Ergebnis	Logisch notwendige Schlussfolgerungen	Wahrscheinliche Verallgemeinerungen
Beispiel	„Alle Menschen sind sterblich. Sokrates ist ein Mensch. → Sokrates ist sterblich."	„Diese 10.000 Kunden kauften nach A auch B. → Empfehle B nach Kauf von A."
Einsatz in DS	Regelbasierte Systeme, Constraint-Solving	Machine Learning, Data Mining

Data Science ist primär induktiv: Muster werden aus Daten extrahiert — keine vollständige Gewissheit, aber probabilistisch fundierte Aussagen.

Der KDD-Prozess

Knowledge Discovery in Databases (KDD) beschreibt den vollständigen Weg von Rohdaten zum Wissen:



- **Selection:** Auswahl relevanter Daten aus dem Gesamtbestand
- **Preprocessing:** Fehlende Werte, Duplikate, Ausreißer behandeln
- **Transformation:** Merkmale ableiten, Dimensionsreduktion
- **Data Mining:** Algorithmen auf transformierten Daten anwenden
- **Interpretation:** Gefundene Muster fachlich bewerten und validieren

In der Praxis wird der Prozess mehrfach durchlaufen. Neue Erkenntnisse im Mining-Schritt führen häufig zur Überarbeitung der Preprocessing-Schritte.

Data Mining Aufgaben im Überblick

Kategorie	Sub-Aufgabe	Beschreibung	Beispiel
Predictive	Classification	Klasse aus bekannten Attributen vorhersagen	Spam-Filter, Kreditrisiko
Predictive	Regression	Numerischen Wert vorhersagen	Preisvorhersage, Nachfrage
Predictive	Collaborative Filtering	Präferenzen aus ähnlichen Nutzern ableiten	Film-/Produktempfehlungen
Descriptive	Clustering	Ähnliche Objekte ohne Klassen gruppieren	Kundensegmentierung
Descriptive	Association Rules	Häufig gemeinsam auftretende Muster	Marktkorb-Analyse
Descriptive	Sequential Patterns	Zeitlich geordnete Muster	Klickpfad-Analyse
Explorative	Anomaly Detection	Abweichungen von Normalverhalten finden	Betrugserkennung

Classification: Vorhersage mit bekannten Klassen

Prinzip

1. **Trainingsdaten:** Datensatz mit bekanntem Klassen-Attribut
2. **Modelltraining:** Algorithmus lernt Entscheidungsregeln
3. **Testdaten:** Modell auf unbekannte Daten anwenden
4. **Evaluation:** Genauigkeit messen (Accuracy, F1, etc.)

Beispiel: Entscheidungsbaum

```
Alter > 30?
├── Ja → Einkommen > 40k?
│        ├── Ja → Kauf wahrscheinlich ✓
│        └── Nein → Kauf unwahrscheinlich ✗
└── Nein → Kauf unwahrscheinlich ✗
```

Einsatz

- E-Mail: Spam oder kein Spam?
- Medizin: Diagnose aus Symptomen
- Marketing: Kauft dieser Kunde?
- Kredit: Kreditausfall wahrscheinlich?

Algorithmen

- Decision Trees
- Random Forests
- Support Vector Machines
- Neural Networks

Clustering: Gruppierung ohne Klassen

Clustering findet natürliche Gruppen in Daten — ohne vorgegebene Klassen.

Beispiel: Kundensegmentierung

```
Cluster A: Jung, hohes Einkommen, Hochschule
  → Premium-Angebote, digitale Kanäle

Cluster B: Mittleres Alter, mittleres Einkommen
  → Familienprodukte, Newsletter

Cluster C: Senioren, niedriges Einkommen
  → Basistarife, persönliche Beratung
```

Eigenschaften

- kein Klassen-Attribut notwendig
- Anzahl der Cluster oft vorab wählen (k-Means)
- Ergebnisse erfordern fachliche Interpretation

Algorithmen

- k-Means (Partitionierung)
- DBSCAN (Dichtebasiert)
- Hierarchisches Clustering

Association Rule Mining

Findet Produkte oder Ereignisse, die häufig gemeinsam auftreten.

Marktkorb-Analyse

Transaktion	Produkte
T1	p1, p3, p5, p8
T2	p2, p5, p8
T3	p1, p5, p8
T4	p3, p4

Regel: {p5} → {p8} mit Support 75%, Confidence 100%

→ Wer p5 kauft, kauft fast immer auch p8.

Anwendungen

- **Retail:** Platzierung verwandter Produkte
- **Online-Shop:** „Kunden, die X kauften, kauften auch Y"
- **Medizin:** Symptom-Diagnose-Korrelationen

Kennzahlen

- **Support:** Wie oft tritt die Regel auf?
- **Confidence:** Wie zuverlässig ist die Regel?
- **Lift:** Wie viel besser als Zufall?

Text Mining und Graph Mining

Text Mining

Klassische Data-Mining-Methoden auf Textdaten:

- **Web-Seiten clustern:** Ähnliche Themen automatisch gruppieren
- **Seiten klassifizieren:** Spam, Kategorie, Sentiment
- **Named Entity Recognition:** Personen, Orte, Unternehmen
- **Topic Modeling:** Latente Themenstrukturen finden

Herausforderung: Texte sind hochdimensional und dünn besetzt.

Graph Mining

Muster in Netzwerkdaten erkennen:

- **Community Detection:** Gruppen in sozialen Netzwerken
- **Link Prediction:** Wer wird mit wem verbunden?
- **Anomalie-Erkennung:** Ungewöhnliche Verbindungsmuster
- **Pagerank:** Zentrale Knoten im Netzwerk

Beispiel: Betrugserkennung über ungewöhnliche Transaktionsnetzwerke.

Mini-Übung: Mining-Aufgaben klassifizieren

Ordnen Sie die folgenden Anwendungen einer Mining-Kategorie zu: **Classification, Clustering, Regression** oder **Collaborative Filtering**.

Anwendung	Kategorie
Netflix empfiehlt Serien basierend auf ähnlichen Nutzerprofilen	?
Ein Modell schätzt den Verkaufspreis eines Hauses	?
Kunden werden nach Kaufverhalten in Gruppen eingeteilt	?
Eine Bank entscheidet: Kreditantrag genehmigen oder ablehnen?	?

Musterlösung

Anwendung	Kategorie
Netflix empfiehlt Serien basierend auf ähnlichen Nutzerprofilen	**Collaborative Filtering**
Ein Modell schätzt den Verkaufspreis eines Hauses	**Regression**
Kunden werden nach Kaufverhalten in Gruppen eingeteilt	**Clustering**
Eine Bank entscheidet: Kreditantrag genehmigen oder ablehnen?	**Classification**

Data Science ist die Disziplin, die aus Daten belastbares Wissen macht — durch den KDD-Prozess, induktive Lernmethoden und iterative Modellentwicklung. Wer die Aufgabe falsch wählt, erhält richtige Antworten auf die falsche Frage.

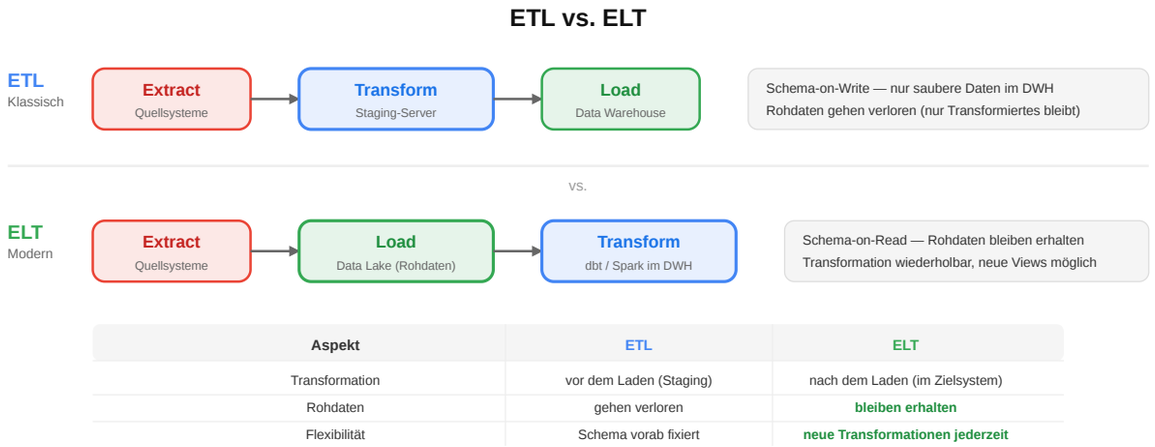
Modul 3

Batch, Streaming und Orchestrierung

Leitfrage: Wie verarbeitet man Daten im Petabyte-Maßstab?

Das klassische Werkzeug: Data Warehouse

Bevor Big Data die Annahmen sprengte, war das **Data Warehouse (DWH)** die Standardantwort für analytische Datenhaltung:



- **ETL:** Daten extrahieren, transformieren, laden — klassisch
- **ELT:** Erst laden (Rohdaten erhalten), dann im Zielsystem transformieren — modern

OLAP vs. OLTP

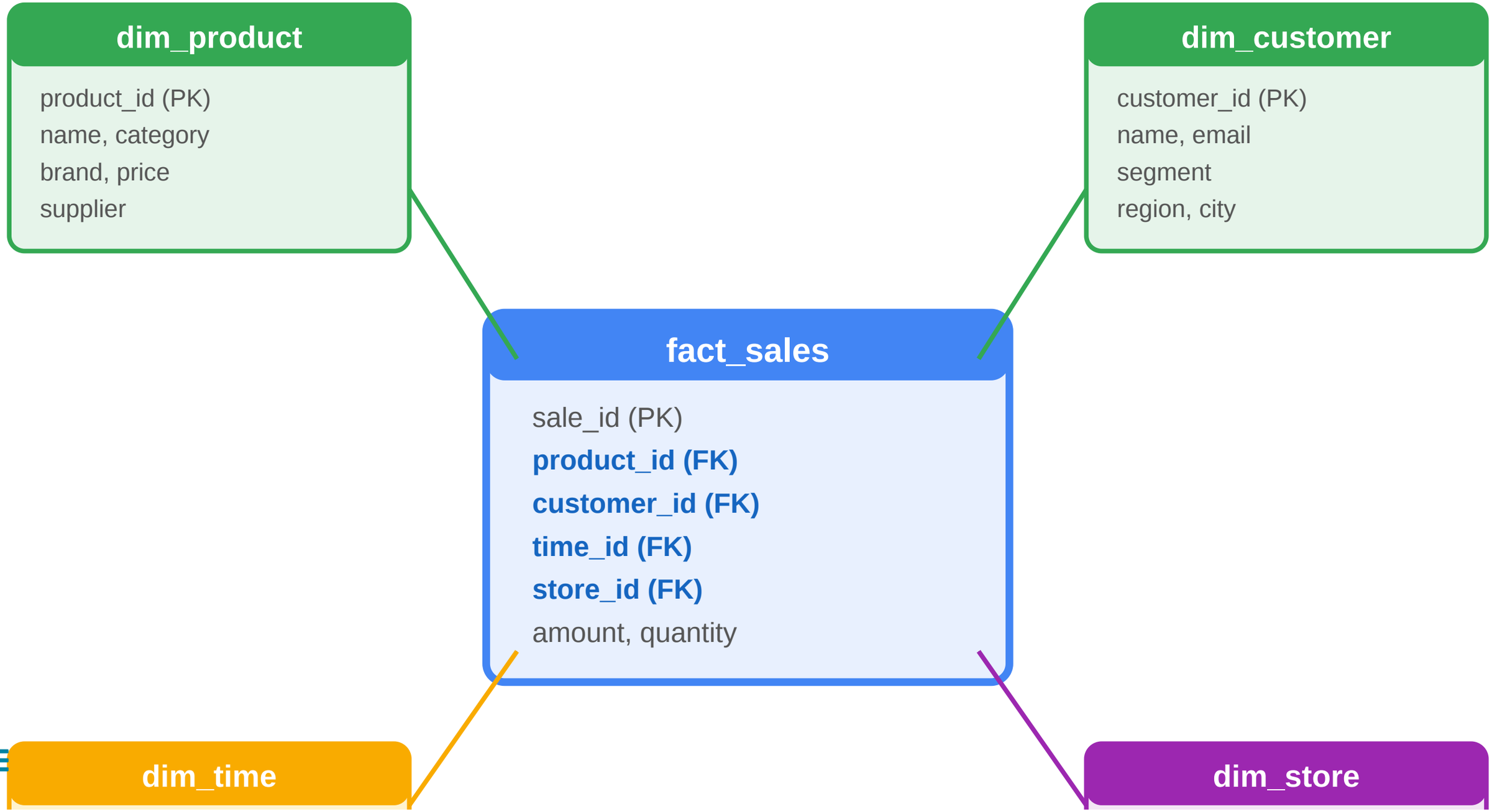
Aspekt	OLTP	OLAP
Zweck	Transaktionen	Analysen
Abfragen	viele, kurz	wenige, komplex
Daten	aktuell	historisch
Beispiel	Bestellsystem	Quartalsbericht

Star Schema

- **Faktentabelle:** Große Akkumulation numerischer Fakten
- **Dimensionstabellen:** Statische Beschreibungen von Entitäten
- **Maßzahlen:** Aggregate über Dimensionen

Star Schema: Datenmodell

Star Schema


dim_product
product_id (PK)
name, category
brand, price
supplier
dim_customer
customer_id (PK)
name, email
segment
region, city
fact_sales
sale_id (PK)
product_id (FK)
customer_id (FK)
time_id (FK)
store_id (FK)
amount, quantity
dim_time
dim_store

Warum das DWH an Grenzen stößt

Die Fünf Vs stellen die Grundannahmen klassischer DWH-Systeme direkt in Frage:

Annahme des DWH	Herausforderung durch Big Data
Daten passen auf einen großen Server	**Volume:** Petabyte-Mengen erfordern Verteilung
Schema ist vorab bekannt	**Variety:** JSON, Logs, Graphen, Streams
ETL läuft nachts, Latenz ist akzeptabel	**Velocity:** Echtzeit-Anforderungen
Quelldaten sind verlässlich	**Veracity:** Systematische Qualitätsprobleme
Analytischer Mehrwert rechtfertigt Aufwand	**Value:** Kosten-Nutzen-Relation bei Rohdaten unklar

Data Lake: Schema-on-Read statt Schema-on-Write

Ein **Data Lake** kehrt das DWH-Prinzip um: Erst speichern, dann strukturieren.

Konzept

- **Schema-on-Read:** Schema wird erst beim Lesen definiert — nicht beim Schreiben
- **Native Formate:** JSON, CSV, Parquet, Avro, Bilder, Audio — alle landen roh
- **Günstige Speicherung:** Object Storage (S3, Azure Blob, GCS) statt teurem RDBMS
- **Flexible Analyse:** Verschiedene Konsumenten definieren ihr Schema auf denselben Rohdaten

Kontrast zum DWH

Aspekt	DWH	Data Lake
Schema	beim Schreiben erzwungen	beim Lesen definiert
Formate	nur strukturiert	alle Formate

Vorteile

Aspekt	Vorteil
Kosten	Object Storage ~10× günstiger als RDBMS
Flexibilität	Neue Analysen ohne Re-ETL
Vollständigkeit	Rohdaten bleiben erhalten — Replay-fähig
Variety	Alle Datenformate in einer Ablage

Risiko: Data Swamp

Ohne Governance wird der Data Lake zum **Data Swamp**: Hunderte Verzeichnisse ohne Dokumentation, inkonsistente Schemas, keine Zugriffsrechte — niemand weiß mehr, welche Daten aktuell und verlässlich sind.

Aspekt	DWH	Data Lake
Speicher	RDBMS (teuer)	Object Storage (günstig)
Änderbarkeit	starr	flexibel

Speaker notes

Der Data Lake entstand als Reaktion auf die Starrheit des DWH: Schema-on-Write bedeutet, dass jede Schemaänderung eine ETL-Umstrukturierung erfordert. Bei schnell ändernden Datenquellen (neue App-Features, neue Event-Typen) ist das nicht praktikabel. Der Data Lake löst das durch Verzögerung: Daten landen erstmal roh, und erst beim Lesen muss sich der Konsument entscheiden, welches Schema er anlegen will. Das klingt elegant, erzeugt aber das Data-Swamp-Problem: ohne Katalog, Lineage und Qualitätsprüfungen verliert sich der Überblick schnell.

Das Data-Swamp-Problem

Ein Data Lake ohne Governance ist nur ein teurer Papierkorb.

Typische Symptome

- Hunderte Verzeichnisse ohne Beschreibung
- Niemand weiß, welche Dateien noch aktuell sind
- Schemas inkonsistent zwischen Erzeuger und Konsument
- Keine Zugriffsrechte → jeder liest alles (oder nichts)
- Duplikate und veraltete Snapshots häufen sich

Ursachen

- Kein Data Catalog
- Keine Schema Registry
- Kein Lifecycle-Management (Retention)
- Fehlende Ownership

Gegenmaßnahmen

Problem	Lösung
Orientierung	Data Catalog (DataHub, Atlas)
Schema-Chaos	Schema Registry (Confluent)
Datenqualität	Automatisierte Quality Gates
Zugriffsrechte	Row-/Column-Level Security
Veraltete Daten	Retention Policies, Partitionierung

> *„Ein Data Swamp ist kein technisches, sondern ein organisatorisches Problem."*

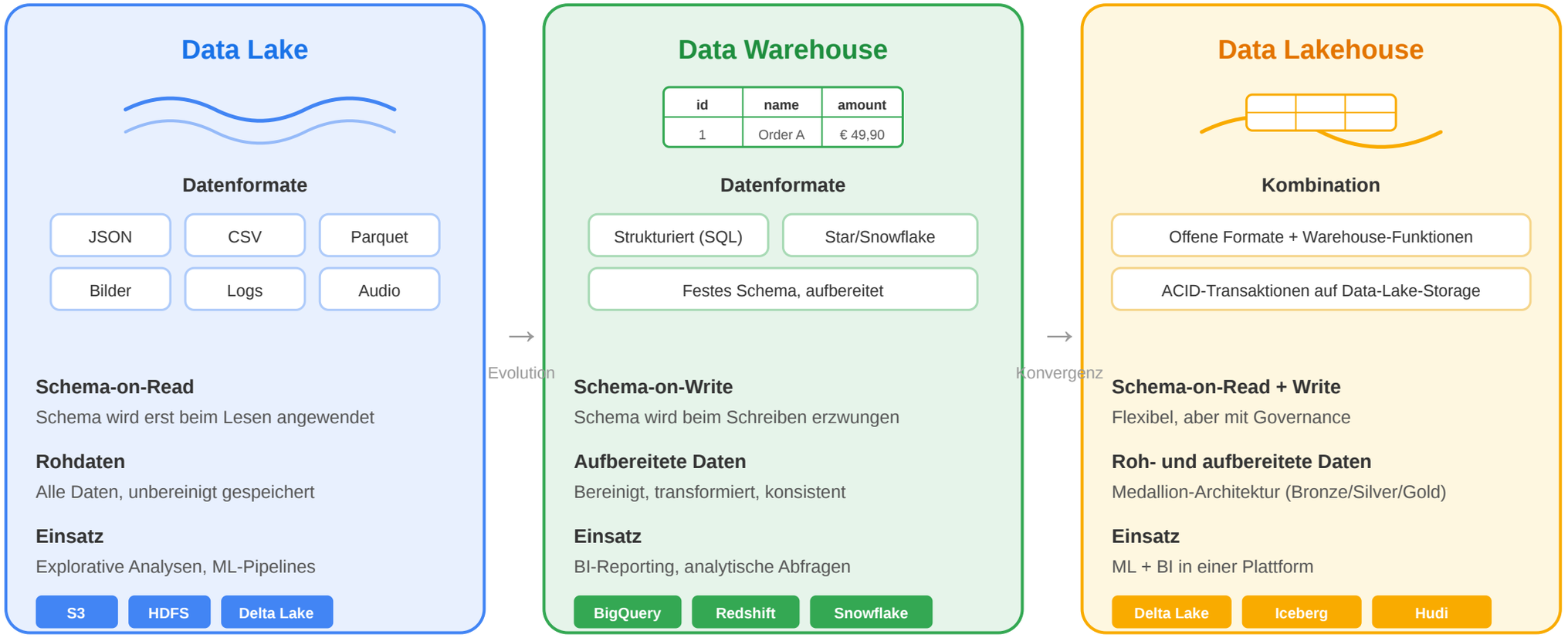
Die gleichen Governance-Maßnahmen aus Modul 4 gelten hier — aber im Data Lake sind sie noch wichtiger, weil die Struktur nicht vom Schema erzwungen wird.

Speaker notes

Der Data Swamp ist eine der häufigsten Fallen bei der Data-Lake-Einführung. Das Problem ist systemisch: Wer Daten hineinkippt, trägt keine Kosten — wer sie später lesen will, trägt alle Kosten. Das führt zu asymmetrischen Anreizen: Produzenten schreiben, Konsumenten kämpfen. Die Lösung ist Governance-First: Kein Datensatz landet im Lake ohne Catalog-Eintrag, Schema-Dokumentation und nominellen Eigentümer. Das klingt bürokratisch, ist aber die einzige Möglichkeit, den Lake langfristig nutzbar zu halten.

Lakehouse: Das Beste aus beiden Welten

Data Lake vs. Data Warehouse vs. Data Lakehouse



Trend: Data Lakehouse vereint Flexibilität des Data Lake mit der Struktur des Data Warehouse

Das **Lakehouse**-Muster vereint die Kosteneffizienz des Data Lake mit der Verlässlichkeit des Data Warehouse.

Vergleich der drei Paradigmen

Implementierungen

Eigenschaft	DWH	Data Lake	Lakehouse
Speicherkosten	hoch	niedrig	niedrig
ACID-Transaktionen	✓	✗	✓
Schema-Enforcement	strikt	keins	wählbar
BI / SQL-Zugriff	✓	eingeschränkt	✓
ML / Rohdaten	✗	✓	✓
Time Travel	✗	✗	✓
Offene Formate	✗	✓	✓

Wie es funktioniert

Offene **Tabellenformate** legen eine Transaktionsschicht auf Object Storage:

- ACID-Transaktionen auf Parquet-Dateien
- Snapshot-basiertes **Time Travel** (rückwirkende Abfragen)
- Schema-Evolution ohne Breaking Changes

Format	Herkunft	Besonderheit
Delta Lake	Databricks (2019)	Transaction Log als JSON
Apache Iceberg	Netflix (2018)	Offenes Spec, breite Unterstützung
Apache Hudi	Uber (2016)	Upserts/Deletes auf Object Storage

Alle drei implementieren dasselbe Konzept: ein **Manifest / Transaction Log** neben den Parquet-Dateien, das Commits, Deletes und Snapshots verwaltet.

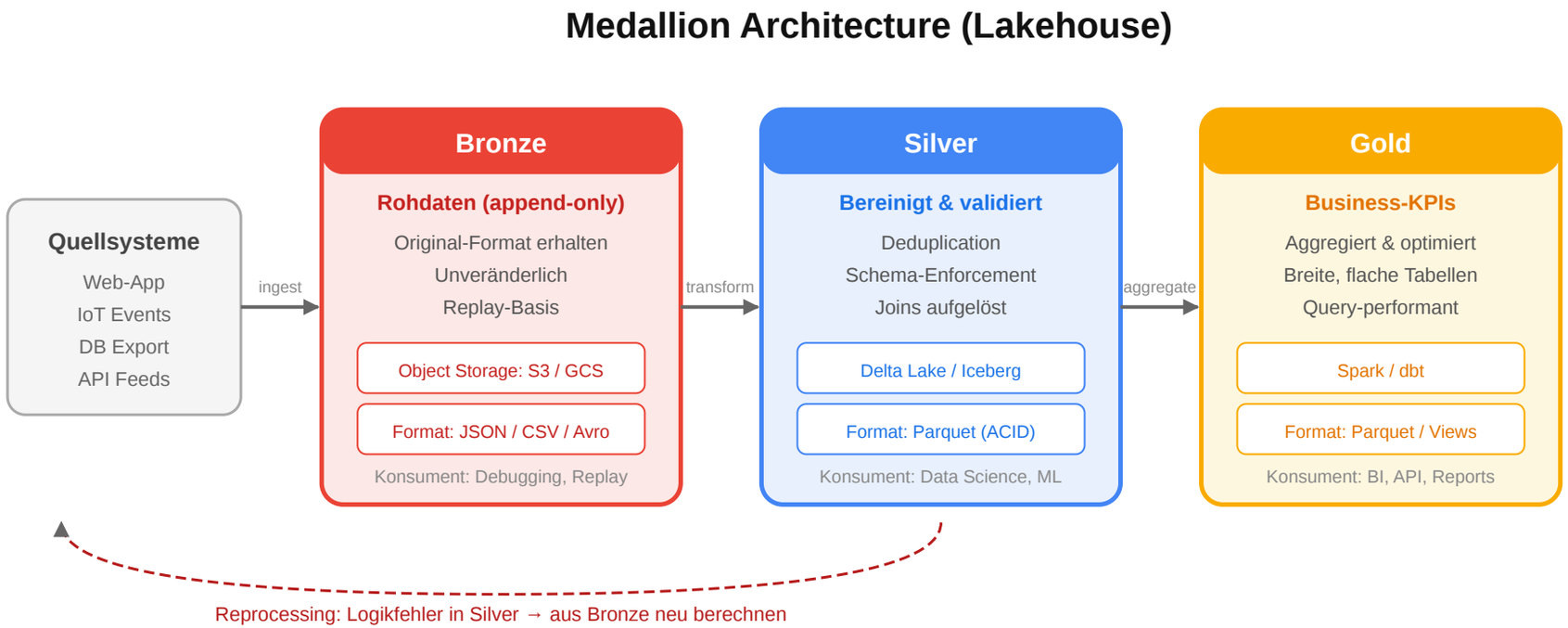
Warum das wichtig ist

- DSGVO **Recht auf Löschung**: Delete auf Object Storage war vorher nicht möglich
- **Upserts:** Daten korrigieren ohne kompletten Neulauf
- **Time Travel:** `SELECT * FROM tabelle AT VERSION 42`

Speaker notes

Das Lakehouse-Konzept wurde 2020 von Databricks geprägt, aber die Idee ist älter: Netflix hatte 2018 Apache Iceberg entwickelt, um genau diese Lücke zu schließen. Das Problem: Object Storage wie S3 ist append-only — man kann Dateien hinzufügen oder löschen, aber nicht überschreiben. Für ACID-Transaktionen und DSGVO-konforme Löschungen braucht man eine Abstraktionsschicht darüber. Das ist die Rolle von Delta/Iceberg/Hudi. Sie verwalten ein Transaction Log neben den eigentlichen Parquet-Dateien und emulieren damit Transaktionen auf einem Dateisystem, das keine kennt.

Lakehouse: Medallion Architecture



Die **Medallion Architecture** strukturiert den Datenfluss im Lakehouse in drei Schichten:

```
Object Storage (S3 / GCS / ADLS)
 ├──── bronze/    ← Rohdaten wie angeliefert, append-only
 ├──── silver/    ← Transformiert, bereinigt, dedupliziert
 └──── gold/      ← Aggregiert, für Analysen optimiert
```

Schicht	Inhalt	Konsumenten
Bronze	Rohdaten, unveränderlich, vollständig	Debugging, Replay, Reprocessing
Silver	Bereinigt, validiert, joins aufgelöst	Data Scientists, Feature Engineering
Gold	Aggregierte KPIs, Business-Metriken	BI-Dashboards, APIs, Reports

Die Schichten bauen aufeinander auf — jede kann bei Logikfehlern neu aus der vorherigen berechnet werden. Bronze bleibt die unveränderliche Wahrheitsquelle.

Speaker notes

Die Medallion Architecture ist eine Konvention, kein Standard — aber sie hat sich als sehr praktisch erwiesen. Bronze entspricht dem "Landing Zone"-Konzept im DWH, Silver der Staging-Schicht, Gold den Data Marts. Der entscheidende Unterschied: Im Lakehouse sind alle Schichten auf demselben Object Storage abgelegt (nur in verschiedenen Verzeichnissen), während im klassischen DWH verschiedene Systeme involviert wären. Das macht Backfilling und Reprocessing erheblich einfacher: Wenn Silver-Logik geändert wird, liest man einfach Bronze neu durch und schreibt Silver neu.

Spaltenorientierte Speicherung: Warum Parquet?

Spaltenorientiertes Speicherlayout

Zeilenbasiert (RDBMS)

id	name	preis	kat.
1	Schuh	49.90	Mode
2	Hemd	29.90	Mode
3	Buch	14.90	Lit.

Auf Disk gespeichert als:

`[1,Schuh,49.90,Mode] [2,Hemd,29.90,Mode] [3,Buch,14.90,Lit.]`

Abfrage: `SELECT AVG(preis)`

✗ id mitlesen | ✗ na | ✓ preis lesen | ✗ kat.

→ 4× so viel I/O wie nötig

Spaltenbasiert (Parquet)

id	name	preis	kat.
1	Schuh	49.90	Mode
2	Hemd	29.90	Mode
3	Buch	14.90	Lit.

Auf Disk gespeichert als:

`preis: [49.90, 29.90, 14.90, ...] kat: [Mode, Mode, Lit., ...]`

Abfrage: `SELECT AVG(preis)`

✓ Nur preis-Spalte lesen — alle anderen Spalten überspringen

✓ Gleiche Werte komprimierbar: [Mode×2, Lit.×1] → Run-Length

Analytische Abfragen lesen selten alle Spalten — columnar Formate sind dafür optimiert.

Zeilenbasiert (RDBMS)

```
| id | name  | preis | kategorie |
|----|-------|-------|-----------|
| 1  | Schuh | 49.90 | Mode      |
| 2  | Hemd  | 29.90 | Mode      |
| 3  | Buch  | 14.90 | Literatur |
```

Auf Disk gespeichert als:

```
[1,Schuh,49.90,Mode] [2,Hemd,29.90,Mode] ...
```

`SELECT AVG(preis)` muss alle anderen Felder trotzdem lesen.

Gut für: INSERT/UPDATE, Lesen einzelner Zeilen (OLTP) **Schlecht für:** Aggregationen über viele Zeilen (OLAP)

Spaltenbasiert (Parquet)

Auf Disk gespeichert als:

```
id:        [1, 2, 3, ...]
name:      [Schuh, Hemd, Buch, ...]
preis:     [49.90, 29.90, 14.90, ...]
kategorie: [Mode, Mode, Literatur, ...]
```

`SELECT AVG(preis)` liest **nur** die Preisspalte.

Vorteile:

- **Kompression:** Ähnliche Werte nebeneinander → Snappy/ZSTD: 5–10×
- **Predicate Pushdown:** `WHERE kategorie='Mode'` überspringt irrelevante Row Groups
- **Vektorisierung:** CPU-SIMD-Instruktionen auf homogenen Arrays

Nicht für: Updates einzelner Zeilen (ganze Datei-Chunks neu schreiben)

Speaker notes

Die Intuition hinter columnar Storage: Eine analytische Abfrage wie `SELECT AVG(preis) FROM produkte WHERE kategorie = 'Mode'` braucht nur zwei Spalten — preis und kategorie. Bei row-orientierten Formaten muss der Storage-Layer trotzdem alle anderen Spalten lesen, weil sie physisch danebenstehen. Bei columnar Formaten liegen preis-Werte hintereinander, kategorie-Werte hintereinander. Das ermöglicht zwei Optimierungen: Erstens, nur relevante Spalten laden (I/O-Reduktion). Zweitens, ähnliche Werte komprimieren besser (z.B. "Mode, Mode, Mode, ..." ist trivial zu komprimieren). Und drittens: modere CPUs können 16–32 Werte parallel mit SIMD-Instruktionen verarbeiten, was bei homogenen Spalten-Arrays direkt möglich ist.

Apache Parquet: Aufbau

Apache Parquet ist das Standard-Dateiformat für analytische Daten im modernen Data Stack:

Dateistruktur

```
Parquet-Datei
├── Row Group 1 (128 MB default)
│   ├── Column Chunk: id
│   │   ├── Page 1 (komprimiert)
│   │   └── Page 2 (komprimiert)
│   ├── Column Chunk: preis
│   └── Column Chunk: kategorie
├── Row Group 2
│   └── ...
└── Footer
    ├── Schema
    ├── Row-Group-Statistiken
    └── Min/Max pro Column Chunk
```

Der **Footer** enthält Min/Max-Werte pro Column Chunk → Parquet kann Row Groups **ohne Lesen überspringen** (Predicate Pushdown).

Vorteile im Überblick

Feature	Vorteil
Columnar	Nur relevante Spalten lesen
Kompression	Snappy / ZSTD: oft 5–10× kleiner als CSV
Predicate Pushdown	Row Groups überspringen ohne Scan
Schema eingebettet	Selbstbeschreibend, kein Katalog nötig
Nested Types	Lists, Maps, Structs nativ unterstützt
Offenes Format	Spark, Hive, DuckDB, BigQuery, Pandas

Faustregel: Für analytische Workloads auf Object Storage immer Parquet (oder ORC) — nie CSV oder JSON für große Datensätze.

Kompressionsbeispiel

```
kategorie-Spalte (1 Mio. Zeilen):
"Mode, Mode, Mode, Literatur, Mode, ..."
→ Run-Length Encoding: (Mode×847k), (Literatur×153k)
→ 90 % Platzeinsparung
```

Speaker notes

Parquet wurde 2013 von Twitter und Cloudera entwickelt und ist heute der de-facto Standard für analytische Daten in Data Lakes und Lakehouses. Der entscheidende Mechanismus ist der Footer mit Statistiken: Wenn eine Abfrage `WHERE preis > 1000` stellt und der Footer sagt, dass Row Group 3 nur Preise zwischen 10 und 50 enthält, muss diese Row Group gar nicht gelesen werden. Bei großen Datasets mit guter Partitionierung (z.B. nach Datum) kann das 90 % des I/O sparen. Das ist der Hauptgrund, warum Spark auf Parquet mit S3 oft schneller ist als auf RDBMS-äquivalenten Systemen.

Apache Arrow: Columnar im Arbeitsspeicher

Apache Arrow ist das In-Memory-Pendant zu Parquet — dasselbe columnar Layout, aber für den RAM.

Das Problem ohne Arrow

```
# Python liest Parquet → deserialisiert zu Python-Objekten
df = pd.read_parquet("data.parquet")
# Spark verarbeitet → Java-Heap-Objekte
# Ergebnis nach R? Erneute Serialisierung
```

Jede Systemgrenze erzwingt teures Kopieren und Konvertieren.

Arrow: Ein gemeinsames Speicherlayout

```
Parquet auf Disk (komprimiert)
      ↓ lesen + dekomprimieren
Arrow-Buffer im RAM ← ein Format für alle
      ↓
Python / Spark / R / DuckDB
lesen denselben Buffer (zero-copy)
```

Warum Arrow wichtig ist

Eigenschaft	Vorteil
Columnar	SIMD-Vektorisierung durch CPU
Zero-Copy	Kein Serialisierungs-Overhead zwischen Prozessen
Sprachübergreifend	Python, Java, C++, R, Julia — ein Puffer
IPC / Flight	Hochperformante Datenübertragung zwischen Diensten

Parquet + Arrow: Das Standard-Paar

Format	Wo	Rolle
Parquet	Disk / Object Storage	Komprimiert, persistent
Arrow	RAM / In-Process	Schnell, interoperabel

Beim Lesen: Parquet → Arrow — dasselbe Schema, minimale Konvertierung. Das ist die Basis für das moderne "zero-copy data sharing" zwischen Python, Spark und DuckDB.

Speaker notes

Arrow wurde 2016 von der Apache Software Foundation gestartet, nachdem der Apache Drill-Entwickler Wes McKinney festgestellt hatte, dass Python (pandas), Java (Spark) und R alle ihre eigenen In-Memory-Repräsentationen für tabellarische Daten hatten. Jede Systemgrenze kostete Serialisierungszeit. Arrow definiert ein einziges binäres Layout für columnar Daten, das alle Sprachen verstehen. Das Ergebnis: Pandas kann einen Arrow-Buffer lesen, Spark kann ihn lesen, R kann ihn lesen — ohne Kopie, ohne Konvertierung. Das Ecosystem-Projekt Apache Arrow Flight erweitert das auf Netzwerk-Transport: hochperformante Datenübertragung zwischen verteilten Systemen, basierend auf demselben columnar Format.

Scaling Up vs. Scaling Out

Scaling Up vs. Scaling Out

Vertical Scaling (Scale Up)

2 CPU
16 GB

→

64 CPU
2 TB RAM
10.000 $/Mo

+ Einfach (ACID funktioniert)
+ Keine verteilte Koordination

− Hardware-Maximum erreicht
− Kosten steigen überproportional

Typisch: RDBMS, DWH

Horizontal Scaling (Scale Out)

Node 1 | Node 2 | Node 3 | Node 4
Node 5 | Node 6 | … N

je 100 $/Mo

+ Linear skalierbar
+ Ausfalltoleranz durch Redundanz

– Verteilte Koordination nötig
– CAP-Theorem: C vs. A

Typisch: HDFS, Spark, Cassandra

Vertical Scaling (Scale Up)

Horizontal Scaling (Scale Out)

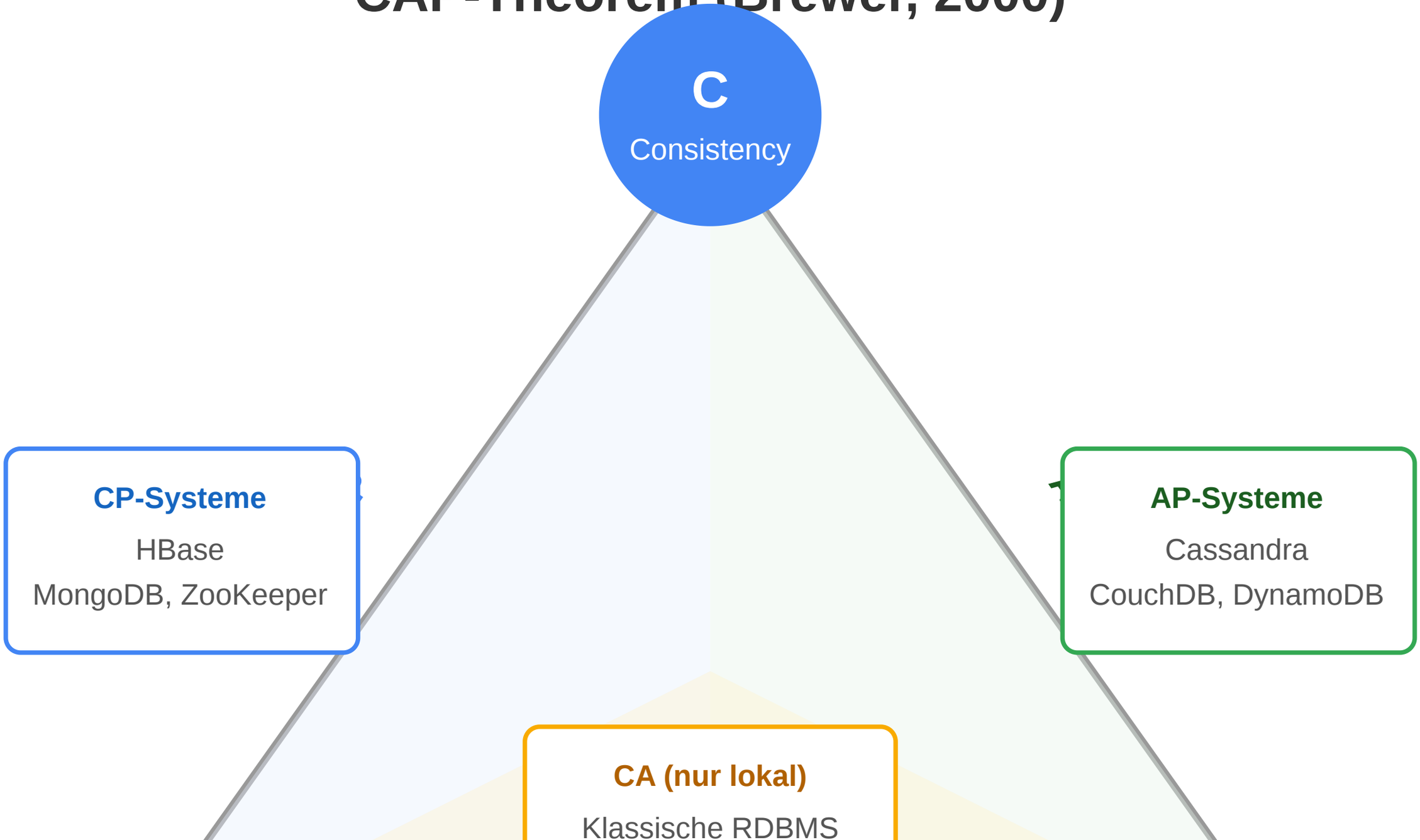
- Größere, leistungsstärkere Einzelmaschine
- RDBMS mit ACID-Garantien funktionieren gut
- Grenze: Hardware-Maximum irgendwann erreicht
- Kosten steigen überproportional

- Viele kleine, günstige Standard-Maschinen
- Jenseits eines Schwellwerts günstiger als Scale Up
- Erfordert verteilte Algorithmen und Koordination
- Ausfalltoleranz durch Redundanz

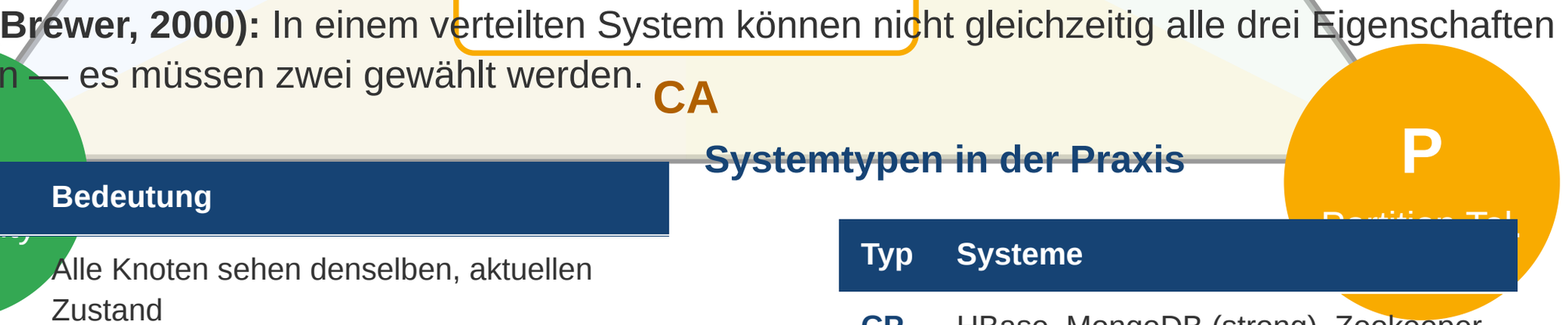
CAP-Theorem: Die Grenzen verteilter Systeme



CAP-Theorem (Brewer, 2000)



CAP-Theorem (Brewer, 2000): In einem verteilten System können nicht gleichzeitig alle drei Eigenschaften garantiert werden — es müssen zwei gewählt werden.



Eigenschaft	Bedeutung
Consistency (C)	Alle Knoten sehen denselben, aktuellen Zustand
Availability (A)	Jede Anfrage erhält eine (möglicherweise veraltete) Antwort
Partition Tolerance (P)	System funktioniert auch bei Netzwerkausfall zwischen Knoten

In der Praxis: **P muss immer gegeben sein**. Die Wahl liegt zwischen **C** und **A**.

Systemtypen in der Praxis

Typ	Systeme
CP	HBase, MongoDB (strong), Zookeeper
AP	Cassandra, CouchDB, DynamoDB
CA	Klassische RDBMS (nur ohne Partition)

Der Paradigmenwechsel: SWAPPING zu CLUSTERING

Traditionell: SWAPPING

```
Disk → RAM → Cache → Register
```

- Daten werden Block für Block von Disk geladen
- Einzelne Maschine verarbeitet sequenziell
- Google indiziert ~10 Mrd. Seiten = 200 TB
- Bei 50 MB/s Bandbreite: **40 Tage** nur zum Lesen

Selbst der schnellste Einzelrechner ist zu langsam für Petabyte-Datenmengen.

Big Data: CLUSTERING

```
[Node 1: 10 GB Chunk] ←→ Computation
[Node 2: 10 GB Chunk] ←→ Computation
[Node 3: 10 GB Chunk] ←→ Computation
         ... 1000 Nodes
```

- Daten in Chunks aufteilen, auf Nodes verteilen
- 200 TB / 1000 Nodes = **200 GB pro Node**
- Computation kommt zu den Daten

Data Locality: Code wird dorthin geschickt, wo die Daten liegen.

Netzwerk und Ausfalltoleranz im Cluster

Netzwerk-Bandbreite

- Innerhalb eines Racks: **1 Gbps**
- Rack-übergreifend (Backbone): **2–10 Gbps**
- Daten-Reshuffling ist teuer: 10 TB bei 1 GB/s = **2,8 Stunden**
- Faustregel: **Reshuffling minimieren**

Ausfalltoleranz

- Bei **1000 Servern**: statistisch ~1 Ausfall pro Tag
- Bei **1.000.000 Maschinen** (Google): ~1000 Ausfälle täglich
- Ausfälle sind keine Ausnahmen — sie sind der **Normalzustand**

Konsequenz:

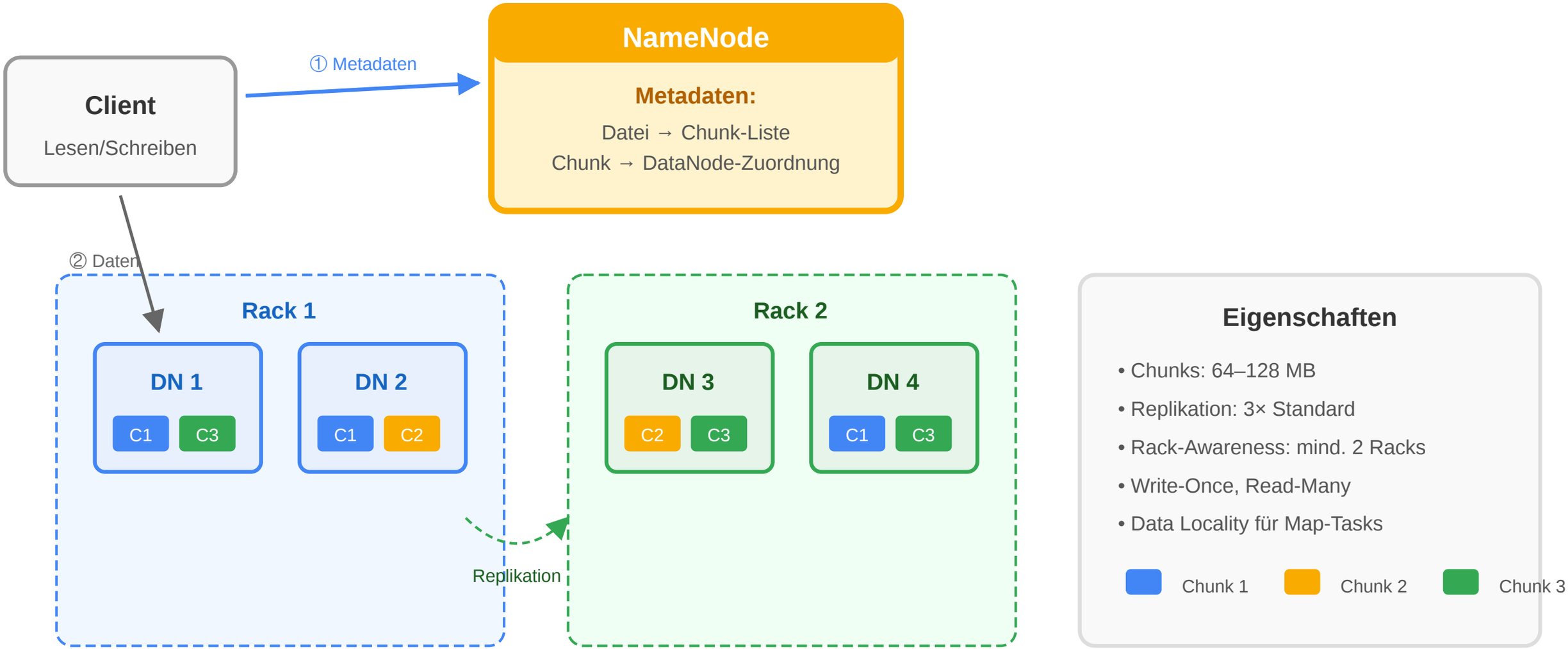
- Daten mehrfach replizieren
- Berechnungen bei Ausfall automatisch neu starten
- Kein Job darf an einem einzelnen Knoten hängen

Design for Failure: Verteilte Systeme müssen von Anfang an auf Ausfälle ausgelegt sein — nicht nachträglich abgesichert werden.

HDFS: Verteiltes Dateisystem

Das Hadoop Distributed File System (HDFS) implementiert das Clustering-Paradigma:

HDFS — Hadoop Distributed File System



Eigenschaften

- Chunks: 64–128 MB
- Replikation: 3× Standard
- Rack-Awareness: mind. 2 Racks
- Write-Once, Read-Many
- Data Locality für Map-Tasks

Chunk 1 Chunk 2 Chunk 3

MapReduce: Berechnung auf verteilten Daten

Google's Programmiermodell für verteilte Datenverarbeitung — heute Grundlage von Hadoop:

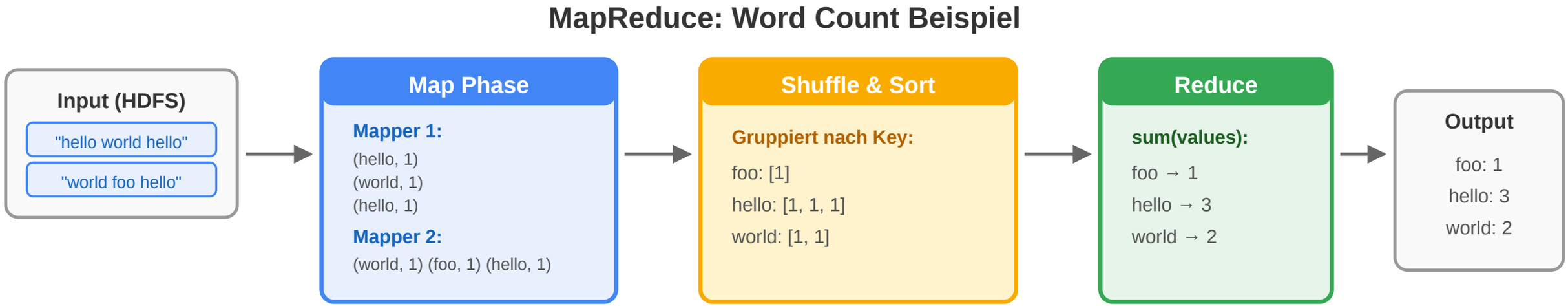
Phasen

1. **Map:** Für jedes <key, value>-Paar: erzeugt eine Menge von <k', v'>-Paaren
2. **Shuffle & Sort:** Intermediäre Paare werden nach Key gruppiert und sortiert
3. **Reduce:** Für jede Gruppe von <k', v'>-Paaren: berechnet ein Aggregat

Effizienz-Prinzipien

- **Streaming through data:** Sequenzielles Lesen ohne Disk Seeks
- **Data Locality:** Map-Tasks laufen auf dem Knoten mit dem jeweiligen Chunk
- **Automatische Fehlerbehandlung:** Fehlgeschlagene Tasks werden neu gestartet

MapReduce Beispiel: Wörter zählen



map(key, value):

```
for word in value.split():
  emit(word, 1)
```

reduce(key, values):

```
emit(key, sum(values))
```

Effizienz-Prinzipien:

1. Data Locality — Map läuft auf dem Chunk-Node
2. Streaming — Sequenzielles Lesen, keine Seeks
3. Parallelisierung — alle Mapper gleichzeitig
4. Fehlertoleranz — Tasks automatisch neu starten

```python
# Pseudo-Code: MapReduce Wortfrequenz
def map_fn(key, value):
    # value = Zeile des Dokuments
    for word in value.split():
        emit(word, 1)

def reduce_fn(key, values):
```

```
    # key = Wort, values = [1, 1, 1, ...]
    emit(key, sum(values))
```



MapReduce als funktionales Paradigma

MapReduce ist keine Hadoop-Erfindung — es ist ein Muster aus der **funktionalen Programmierung**:

Map: Transformation

Wende eine Funktion auf jedes Element einer Collection an:

```
preise  = [10.0, 25.0, 8.5]
doubled = list(map(lambda x: x * 2, preise))
# → [20.0, 50.0, 17.0]
```

Jedes Element ist **unabhängig** — beliebig parallelisierbar.

Reduce: Aggregation

Fasse eine Collection auf einen Wert zusammen:

```
from functools import reduce
total = reduce(lambda a, b: a + b, preise)
# → 43.5
```

Warum das skaliert

```
1000 Knoten, je 1 GB Chunk:

[Chunk 1] → map() → [k', v'] ──┐
[Chunk 2] → map() → [k', v'] ──┤ → reduce() → Ergebnis
[Chunk 3] → map() → [k', v'] ──┘
```

Schlüsseleigenschaft:

- Map-Phase: vollständig parallel, kein Knoten kennt andere
- Reduce-Phase: Aggregation über Schlüssel — nur minimales Reshuffling nötig

Skalierbarkeit aus zwei Garantien:

1. `map` ist **nebeneffektfrei** — keine gemeinsamen Seitenzustände

Dasselbe in SQL

```sql
SELECT kategorie, AVG(preis)  -- Reduce
FROM produkte
GROUP BY kategorie              -- Shuffle + Group
-- implizites Map: alle Zeilen durchlaufen
```

2. `reduce` ist **assoziativ** — `reduce(a, reduce(b, c)) = reduce(reduce(a, b), c)` → Teilergebnisse kombinierbar

Jede Funktion, die diese Garantien erfüllt, lässt sich auf beliebig vielen Knoten parallel ausführen.

Speaker notes

Das MapReduce-Paradigma geht auf LISP (1958) zurück — `map` und `reduce` (dort `fold`) sind funktionale Grundoperationen. Google hat 2004 erkannt, dass diese Abstraktion exakt die richtige ist, um verteilte Berechnungen zu modellieren. Der Trick ist die Nebeneffektfreiheit: Wenn eine Funktion nur ihre Eingabe liest und keine globalen Zustände verändert, kann das Framework beliebig viele Kopien davon parallel starten, ohne Koordination zu benötigen. Das ist der Grund, warum MapReduce so elegant skaliert — die Parallelisierung ist nicht ein nachträgliches Engineering-Problem, sondern eine Konsequenz der Abstraktionswahl.

Grenzen von klassischem MapReduce

Googles Original-MapReduce (2004) hat strukturelle Schwächen, die seinen Nachfolgern den Weg bereiteten:

Problem	Ursache	Konsequenz
Disk-Intensiv	Jede Phase schreibt intermediäre Daten auf HDFS	10–100× langsamer als nötig für iterative Algorithmen
Zwei-Phasen-Modell	Nur Map + Reduce als Primitive	Komplexe Jobs = viele sequentielle MR-Jobs hintereinander
Kein Iterator-Zustand	Reducer erhält alle Werte auf einmal in Speicher	Joins und Iterationen (ML-Training!) teuer
Java-First	Hoher Boilerplate für einfache Transformationen	Produktivitätsproblem für Analysten
Kein Streaming	Nur Batch	Neue Latenzanforderungen nicht erfüllbar

Speaker notes

Der entscheidende Bottleneck von klassischem MapReduce: ML-Algorithmen wie k-Means oder PageRank sind iterativ — sie brauchen denselben Datensatz 10, 50, 100 Mal. Bei jedem Durchlauf schreibt MapReduce alle intermediären Daten auf HDFS (wegen Ausfalltoleranz), liest sie im nächsten Durchlauf wieder ein. Das sind für eine 100-GB-Datensatz 10 × 100 GB Lesen + Schreiben = 2 TB I/O für 100 Iterationen, obwohl die Daten sich nicht geändert haben. Spark löst das durch In-Memory-Caching: nach dem ersten Lesen bleiben die Daten im RAM der Worker-Prozesse und können ohne Disk-Zugriff wiederverwendet werden.

Von MapReduce zu Apache Spark

Apache Spark (AMPLab Berkeley, 2009) löst das Disk-Bottleneck durch **In-Memory-Verarbeitung**:

Resilient Distributed Datasets (RDD)

```
# Spark RDD (Scala/Python API)
rdd = sc.textFile("hdfs://logs/*.txt")

result = (rdd
  .flatMap(lambda line: line.split())   # map
  .map(lambda word: (word, 1))          # map
  .reduceByKey(lambda a, b: a + b)      # reduce
  .sortBy(lambda x: -x[1]))            # sort

result.saveAsTextFile("hdfs://output/")
```

- **Lazy Evaluation:** Operationen werden erst bei `.save()` / `.collect()` ausgeführt
- **Lineage-Graph:** Bei Ausfall wird nur der betroffene Partition neu berechnet
- **In-Memory-Caching:** `rdd.persist()` hält Daten im RAM für Wiederverwendung

DataFrame API + Catalyst-Optimizer (2015)

```
# Spark DataFrame – SQL-nahe Syntax
from pyspark.sql import functions as F

result = (spark.read.parquet("s3://data/")
  .filter(F.col("kategorie") == "Mode")
  .groupBy("hersteller")
  .agg(F.avg("preis").alias("avg_preis"))
  .orderBy("avg_preis", ascending=False))
```

- **Catalyst:** Algebraische Query-Optimierung (wie SQL-Planer)
- **Tungsten:** CPU- und Speicher-Optimierung, Code-Generierung
- **Unified:** Batch, Streaming, ML (MLlib), Graph (GraphX) in einer API
- **Spark SQL:** Direkte SQL-Abfragen auf DataFrames



Speaker notes

Spark ist der wichtigste Nachfolger von MapReduce und heute das dominierende Framework für verteilte Batch-Verarbeitung. Der Schlüsselvorteil gegenüber MapReduce ist das In-Memory-Caching: Spark kann einen RDD/DataFrame im RAM halten und für mehrere Operationen wiederverwenden, ohne ihn neu zu lesen. Für ML-Algorithmen mit 100 Iterationen bedeutet das 100× weniger I/O. Die DataFrame API (2015) war ein weiterer Quantensprung: Statt Java-Code schreiben Data Scientists Python oder SQL, und der Catalyst-Optimizer generiert einen effizienten physischen Plan. In der Praxis ist Spark auf Parquet-Daten in S3/GCS heute die Standard-Kombination für analytische Batch-Verarbeitung.

Die Evolution des MapReduce-Paradigmas

```
2004  Google MapReduce – Disk-basiert, HDFS, Java
        ↓ Problem: langsam für iterative ML, zu low-level
2009  Apache Spark – In-Memory RDDs, 10–100× schneller
        ↓ Problem: zu low-level für SQL-Analysten
2015  Spark DataFrames + SQL (Catalyst Optimizer)
        ↓ Problem: kein unified Batch + Stream API
2016  Spark Structured Streaming + Apache Flink
        ↓ Problem: Ingenieure schreiben Code, Analysten SQL
2018  dbt (Data Build Tool) – SQL-Transformationen als versionierter Code
        ↓ Problem: Python/SQL-Ökosysteme noch getrennt
2020+ DuckDB, Polars – hochperformante In-Process Engines, Arrow-nativ
```

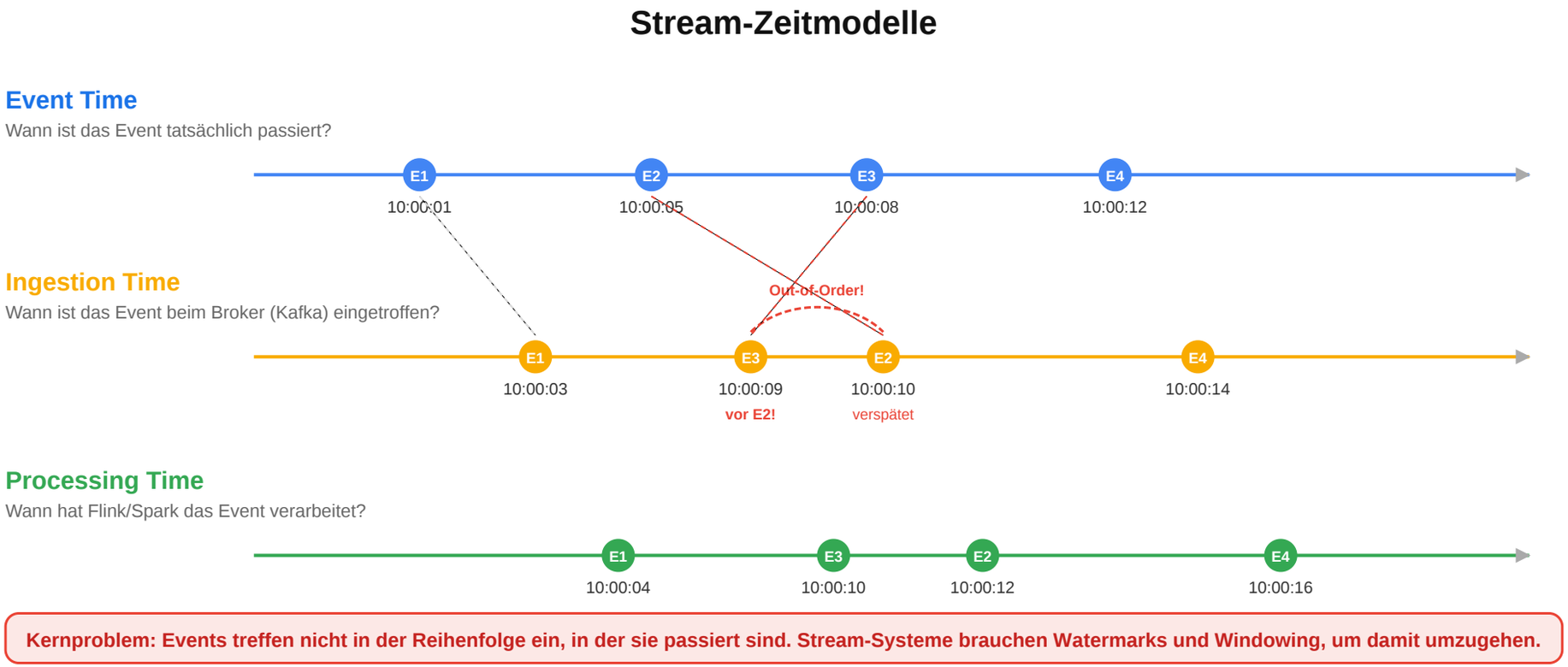
Framework	Stärke	Typischer Einsatz
MapReduce	Einfachheit, Ausfalltoleranz	Legacy Hadoop
Spark	Geschwindigkeit, ML-Integration	Batch + Streaming, Feature Engineering
Flink	Exakte Streaming-Semantik	Echtzeit, stateful Processing
dbt	SQL-Transformationen als Code	Data Warehouse, Lakehouse Transformationen
DuckDB	In-Process OLAP, Arrow-nativ	Analytik ohne Cluster, Notebook-Workloads

Speaker notes

Die Evolution des MapReduce-Paradigmas zeigt ein wiederkehrendes Muster: Jede Generation löst die Bottlenecks der vorherigen und verschiebt das Problem eine Abstraktionsebene höher. MapReduce war disk-bound → Spark löst es durch RAM. Spark war zu low-level für SQL-Nutzer → DataFrames und Spark SQL lösen es. Batch und Stream waren getrennt → Structured Streaming und Flink vereinen sie. Jetzt trennen noch Python-Code (für Ingenieure) und SQL (für Analysten) die Welt → dbt löst es durch SQL-first Transformation. Und DuckDB löst das Cluster-Overhead-Problem für kleinere Datasets: Heute lassen sich 100-GB-Analysen auf einem Laptop in Sekunden durchführen, was früher ein Hadoop-Cluster erfordert hätte.

Stream-Zeitmodelle: Warum Reihenfolge trügt

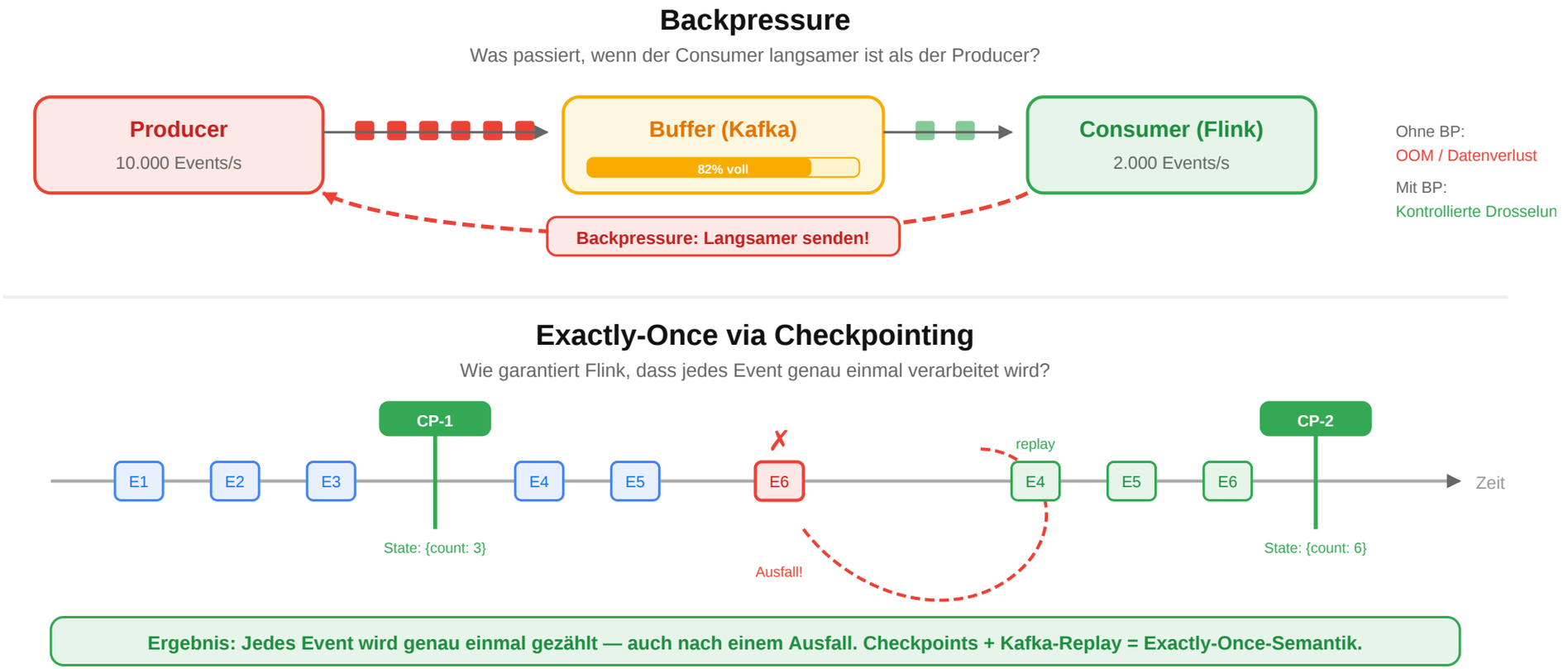
Im Streaming ist Zeit nicht trivial — dasselbe Event hat drei verschiedene Zeitstempel:



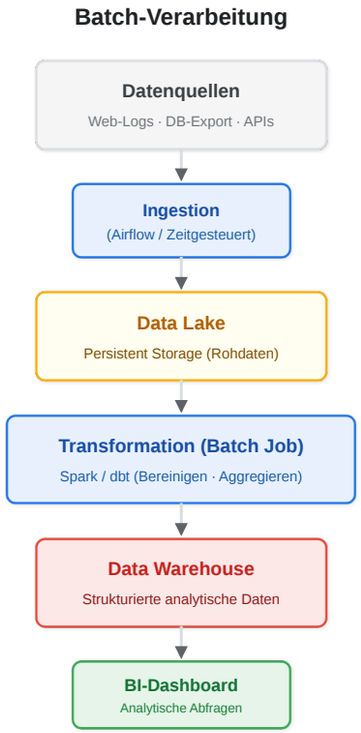
Kernproblem: Events treffen nicht in der Reihenfolge ein, in der sie passiert sind. Stream-Systeme brauchen Watermarks und Windowing, um damit umzugehen.

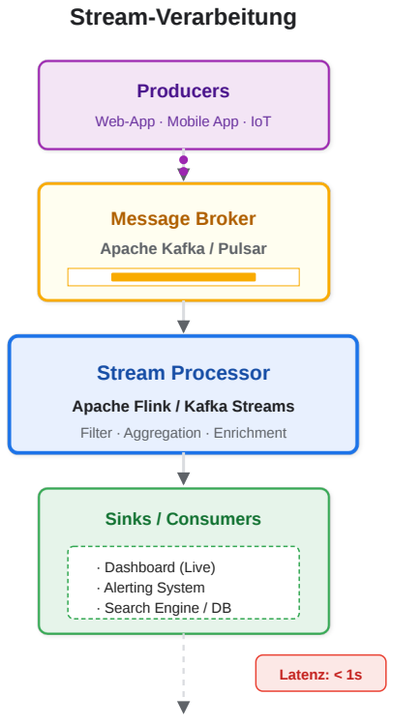
Backpressure und Exactly-Once-Semantik

Zwei zentrale Herausforderungen im Stream Processing:



Batch vs. Streaming: Wann was einsetzen?





Batch-Verarbeitung

- Daten werden gesammelt und periodisch als Block verarbeitet
- **Stärken:** Hoher Durchsatz, Einfachheit, Kosteneffizienz
- **Schwächen:** Hohe Latenz (Minuten bis Stunden)
- **Einsatz:** Reporting, ML-Training, historische Analysen, ETL

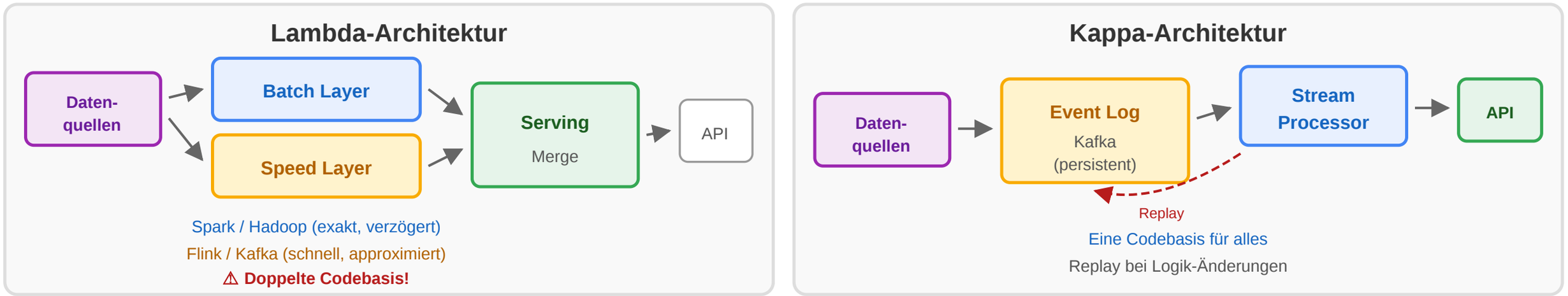
Stream-Verarbeitung

- Jedes Event wird sofort nach Eintreffen verarbeitet
- **Stärken:** Niedrige Latenz (Millisekunden)
- **Schwächen:** Komplexer (Out-of-Order, Duplikate, Checkpoints)
- **Einsatz:** Betrugserkennung, Alerting, Echtzeit-Empfehlungen

Kriterium	Batch	Stream
Latenz	Minuten–Stunden	Millisekunden–Sekunden
Fehlerbehandlung	Neuberechnung des Batchs	Checkpoints, Exactly-once
Engineering-Aufwand	geringer	höher
Wirtschaftlichkeit	günstiger bei großen Mengen	teurer bei hohem Volumen

Lambda- und Kappa-Architektur

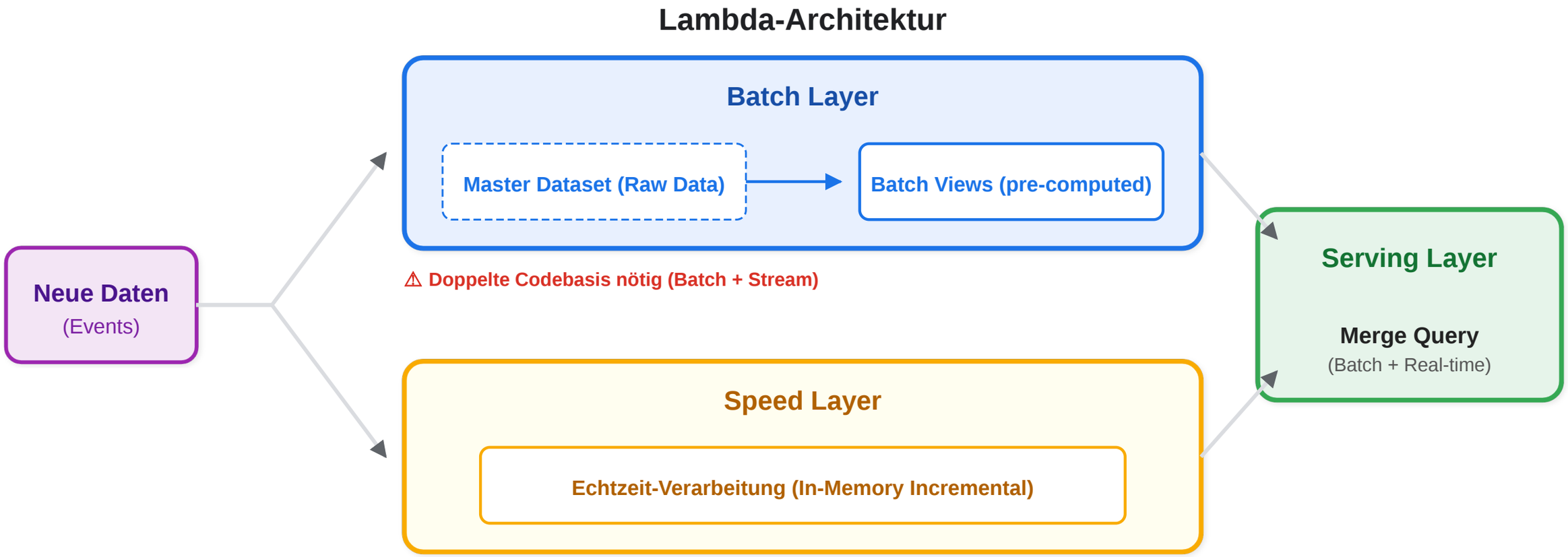
Lambda- vs. Kappa-Architektur



Aspekt	Lambda	Kappa
Verarbeitungspfade	Zwei (Batch + Stream)	Einer (nur Stream)
Codebasis	Doppelt	Einfach
Historische Daten	Batch Layer	Replay aus Event-Log
Betriebsaufwand	Hoch	Geringer
Genauigkeit (historisch)	Exakt	Abhängig von Log-Tiefe

Lambda-Architektur: Batch + Stream kombiniert

Problem: Wie bekommt man historische Vollständigkeit **und** Echtzeit-Ergebnisse?



Kombiniert historische Korrektheit mit minimaler Latenz

Struktur

Konkretes Beispiel: Empfehlungssystem

- **Batch Layer** (Spark, Hadoop): Vollständige historische Neuberechnung periodisch — exakt, aber verzögert
- **Speed Layer** (Flink, Kafka Streams): Nur neue Events seit letztem Batch — sofortige, approximierte Ergebnisse
- **Serving Layer:** Kombiniert beide Pfade für Abfragen

Nachteil — Doppelte Codebasis: Dieselbe Verarbeitungslogik muss für Batch **und** Stream implementiert und synchron gehalten werden.

```
Batch (täglich, Spark):
  Nutzer-Profil aus 90-Tage-Klickhistorie
  → vollständige, exakte Modelle

Speed (Echtzeit, Kafka Streams):
  Letzte 10 Klicks der aktuellen Session
  → schnelle, approximierte Updates

Serving:
  Merge(Batch-Profil, Speed-Delta)
  → Empfehlung
```

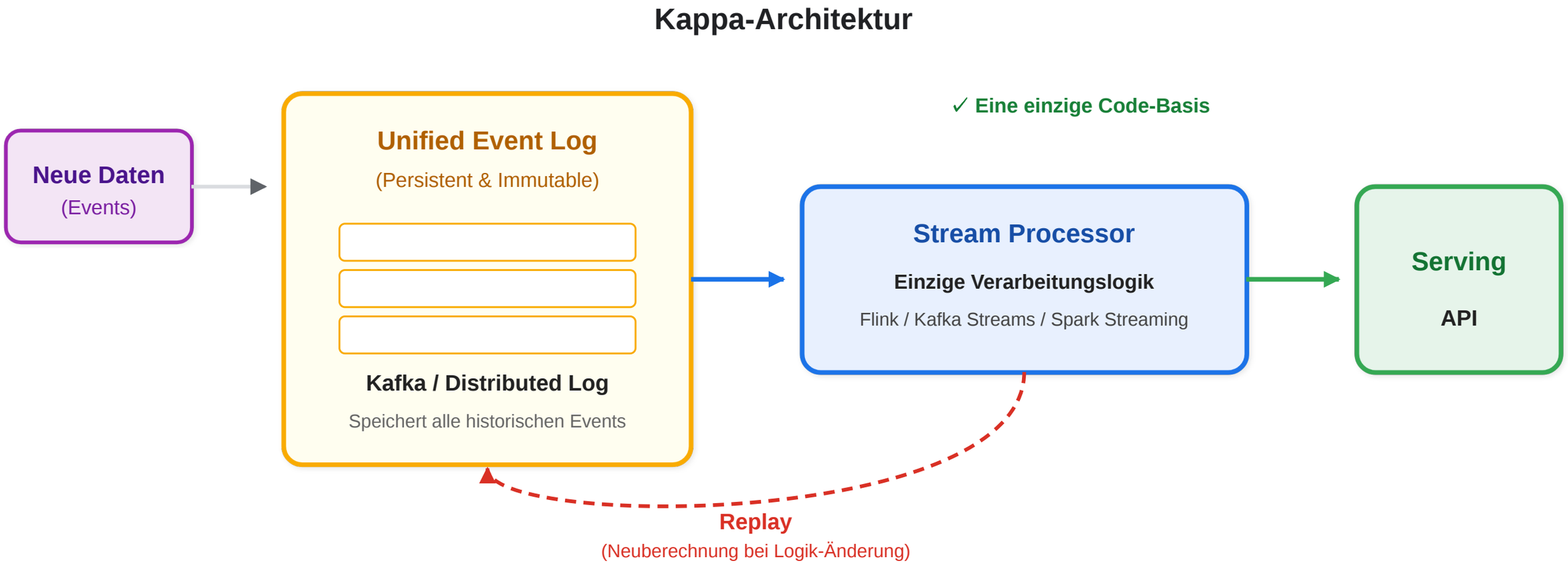
Sinnvoll wenn: ML-Modelle wöchentliches Batch-Retraining auf vollständigen Daten brauchen + trotzdem Echtzeit-Reaktion nötig ist.

Speaker notes

Die Lambda-Architektur wurde 2011 von Nathan Marz geprägt (Autor von Apache Storm). Das Grundproblem ist real: Batch gibt exakte, vollständige Ergebnisse — aber mit Stunden Verzögerung. Stream gibt sofortige Ergebnisse — aber möglicherweise unvollständig (weil nicht alle historischen Daten verarbeitet wurden). Lambda kombiniert beide: Der Batch Layer ist immer "korrekt" (aber veraltet), der Speed Layer ist immer "aktuell" (aber approximiert). Das Serving Layer merged beide. Das klingt elegant, erzeugt aber das "zwei Codebasen"-Problem: Jede Bugfixierung muss in Spark UND in Kafka Streams/Flink gemacht werden. Das führt in der Praxis zu Divergenz.

Kappa-Architektur: Nur Streaming

Idee: Wenn Stream-Systeme robust genug sind — warum noch Batch?



Struktur

- **Alle Daten** als Events im persistenten Log (Kafka)

Chancen und Grenzen

Stärken:

- **Eine Codebasis** — dieselbe Logik für neue Events und historische Daten
- **Replay:** Bei Logik-Änderungen gesamtes Log erneut durchlaufen → konsistente Neuberechnung

Vergleich

Aspekt	Lambda	Kappa
Verarbeitungspfade	zwei	einer
Codebasis	doppelt	einfach
Historische Daten	Batch Layer	Replay aus Log
Betriebsaufwand	hoch	geringer

- Korrekturen durch Replay des gesamten Logs heilbar
- Kein Batch-Framework zu betreiben
- Natürlich für Event-Sourcing-Systeme

Grenzen:

Herausforderung	Ursache
Replay-Kosten	1 TB Log × 100 Partitionen = Stunden Replay
Stateful Complexity	1-Jahres-Aggregat braucht persistenten State
Batch-ML-Training	Manche Modelle brauchen echten vollständigen Scan
Retention	Event-Log muss historisch ausreichend tief sein

Nicht geeignet wenn: Regulatorisches Reporting einen "frozen" Batch-Snapshot verlangt.

Speaker notes

Die Kappa-Architektur wurde 2014 von Jay Kreps (LinkedIn, Kafka-Miterfinder) als Vereinfachung der Lambda-Architektur vorgeschlagen. Die Grundthese: Das "zwei Codebasen"-Problem von Lambda entsteht, weil Batch und Stream als verschiedene Paradigmen behandelt werden. Wenn man Batch als "Streaming mit hoher Latenz" betrachtet, braucht man nur ein Framework. Kafka als persistenter, replay-fähiger Log ist die Schlüsselkomponente: Der gleiche Event-Stream kann einmal live verarbeitet werden und einmal komplett neu durchlaufen werden (Replay), wenn sich die Verarbeitungslogik ändert. Das Problem: Replay dauert bei großen Logs Stunden bis Tage. Und einige Algorithmen (z.B. Gradient-Descent-basierte ML-Modelle) brauchen wirklich mehrfache Scans des gesamten Datensatzes — das ist auf einem reinen Stream schwer zu realisieren.

Lambda vs. Kappa: Wann was einsetzen?

Kriterium	Lambda wählen	Kappa wählen
Latenzanforderung	Minuten akzeptabel für historisch exakte Ergebnisse	Sekunden durchgehend gefordert
ML-Modelle	Wöchentliches Batch-Retraining auf vollständigen Daten	Online-Learning oder tägliche Inkremente ausreichend
Compliance	Frozen Snapshots für Audit erforderlich	Event-Log als Audit-Trail ausreichend
Team-Setup	Verschiedene Teams für Batch und Stream	Ein Team, ein Framework
Log-Retention	Egal (Batch-Daten auf DWH/Lake)	Event-Log muss historisch tief genug sein
Replay-Toleranz	Nicht nötig	Stunden bis Tage akzeptabel

Moderner Trend: Lakehouse + Streaming ersetzt beide Muster zunehmend. Delta Lake / Iceberg erlaubt ACID-Inserts aus Streaming-Jobs → ein einziges Storage-System für Echtzeit und Batch.

```
Kafka (Event Stream)
   ├──── Flink → Delta/Iceberg Streaming Tables   (niedrige Latenz)
   └──── Spark → Delta/Iceberg Batch Tables       (hohe Genauigkeit)
              ↓
        Unified Serving über dieselben Parquet-Dateien
```

Speaker notes

Die Praxis hat sich inzwischen weiterentwickelt: Das Lakehouse-Muster (Delta Lake, Iceberg) löst die Trennung zwischen Batch und Stream auf Storage-Ebene auf. Flink und Spark können beide in dieselben Delta-Tabellen schreiben, mit ACID-Garantien. Das Serving-Layer ist nicht mehr "Merge von zwei separaten Ergebnissen", sondern eine einzige Tabelle, die kontinuierlich aktualisiert wird. In diesem Modell ist Lambda überflüssig — und Kappa vereinfacht sich, weil der persistente State nicht mehr im Stream-Framework leben muss, sondern im Lakehouse-Format auf Object Storage.

Mini-Übung: Batch oder Streaming?

Entscheiden Sie für jedes Szenario: **Batch** oder **Streaming** — und begründen Sie kurz.

Szenario	Batch oder Streaming?
Tagesabschluss-Bericht der Verkaufszahlen	?
Erkennen eines Kreditkartenbetrugs in Echtzeit	?
Wöchentlicher ML-Trainings-Job auf historischen Klick-Daten	?
Alerting bei Serverausfall innerhalb von 5 Sekunden	?
Monatliche Kundensegmentierung für E-Mail-Kampagne	?

Musterlösung

Szenario	Entscheidung	Begründung
Tagesabschluss-Bericht	**Batch**	Latenz von Stunden akzeptabel, hoher Durchsatz
Kreditkartenbetrug	**Streaming**	Muss in Millisekunden entschieden werden
ML-Trainings-Job	**Batch**	Vollständiger historischer Datensatz nötig
Server-Alerting	**Streaming**	Reaktionszeit < 5 Sekunden erforderlich
Kundensegmentierung	**Batch**	Monatlicher Rhythmus, kein Echtzeit-Bedarf

Batch und Stream sind keine Konkurrenten, sondern komplementäre Werkzeuge. Die Wahl hängt von Latenzbedarf, Durchsatz und Konsistenzanforderungen ab — nicht von technischer Mode.

Modul 4

Datenqualität und Governance

Leitfrage: Was macht eine robuste Datenplattform aus — und warum scheitern datengetriebene Systeme oft nicht an Rechenleistung, sondern an Qualität, Verantwortlichkeiten und fehlender Transparenz?

MapReduce: Effizienz durch Lokalität

Warum ist MapReduce so schnell, obwohl es auf tausenden günstiger Maschinen läuft?

Prinzip	Erklärung
Data Locality	Map-Task läuft auf dem Node, der den Chunk speichert — kein Netzwerktransfer für Eingabedaten
Streaming	Daten werden sequenziell gelesen, keine Disk-Seeks — maximale Bandbreite
Pipelining	Map-Tasks laufen parallel auf allen Nodes — horizontale Skalierung
Fehlertoleranz	Framework re-startet fehlgeschlagene Tasks automatisch auf Replikat

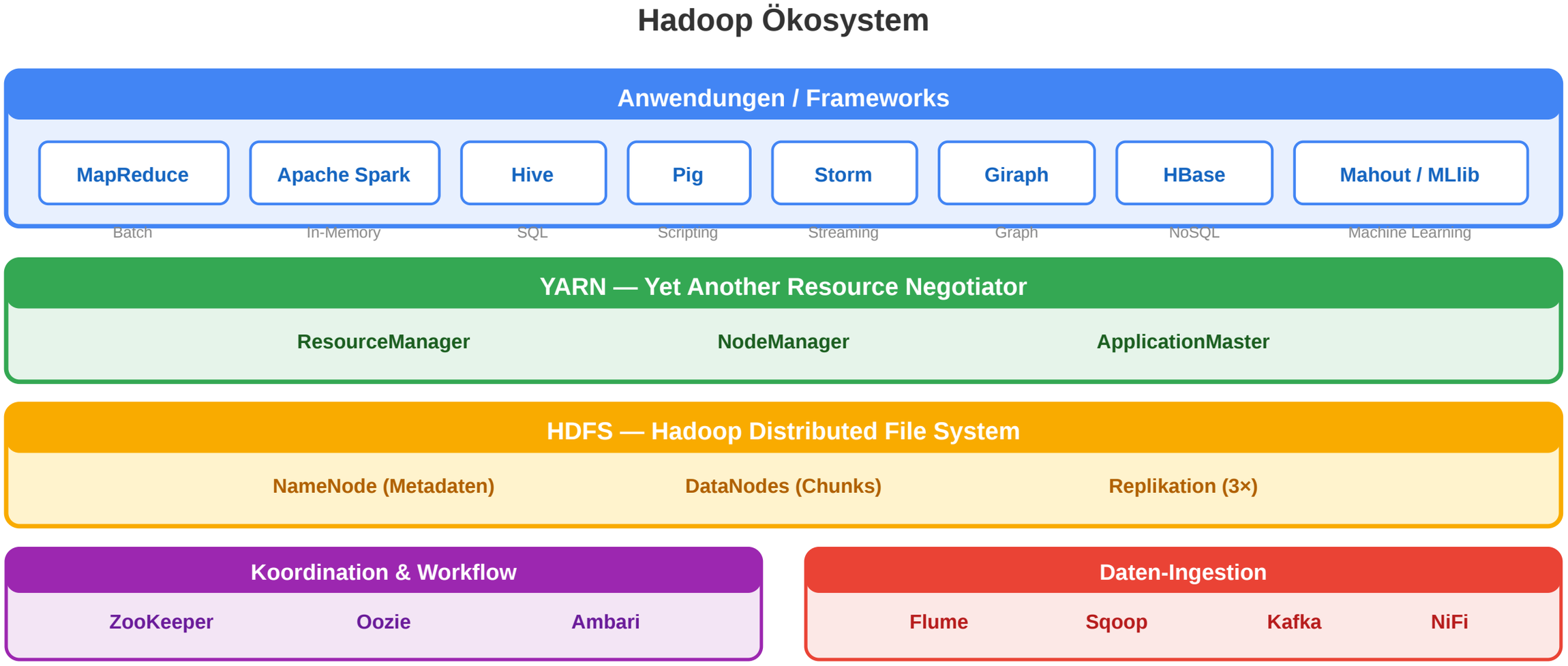
Data Locality Principle

Code ist klein (wenige KB), Daten sind riesig (TB).
Bringe die Berechnung zu den Daten, nicht umgekehrt.

Geeignete Anwendungen:

- Web-Index-Aufbau (Google!)
- Log-Analyse
- ETL-Pipelines
- ML-Feature-Berechnung

Hadoop Ökosystem



Schicht	Technologien	Aufgabe
Anwendungen	MapReduce, Spark, Hive, Pig, Storm, Giraph	Datenverarbeitung (Batch, Streaming, Graph, SQL)

Schicht	Technologien	Aufgabe
Ressourcenmanagement	YARN	Zuweisung von CPU, RAM an Anwendungen im Cluster
Speicher	HDFS	Verteilte, redundante Datenhaltung in Chunks

Hadoop ≠ MapReduce: MapReduce ist nur **eine** Anwendung auf Hadoop. Moderne Stacks nutzen meist Spark (In-Memory, 10–100× schneller) auf YARN/HDFS.

Alternativen zum Scale-Out

Master/Slave Replikation

- Originaler GFS-Ansatz
- Multi-Master-Replikation möglich
- **INSERT only** — keine Updates/Deletes
- Keine JOINs → reduziert Query-Zeit
- Lesezugriff skaliert durch Replicas

Einsatz: Leselastige Systeme, Content-Delivery

In-Memory-Datenbanken

- Gesamte Datenbank im RAM
- SAP HANA, Oracle In-Memory, Redis
- Extrem geringe Latenz
- Teuer, aber ideal für Echtzeitanalysen

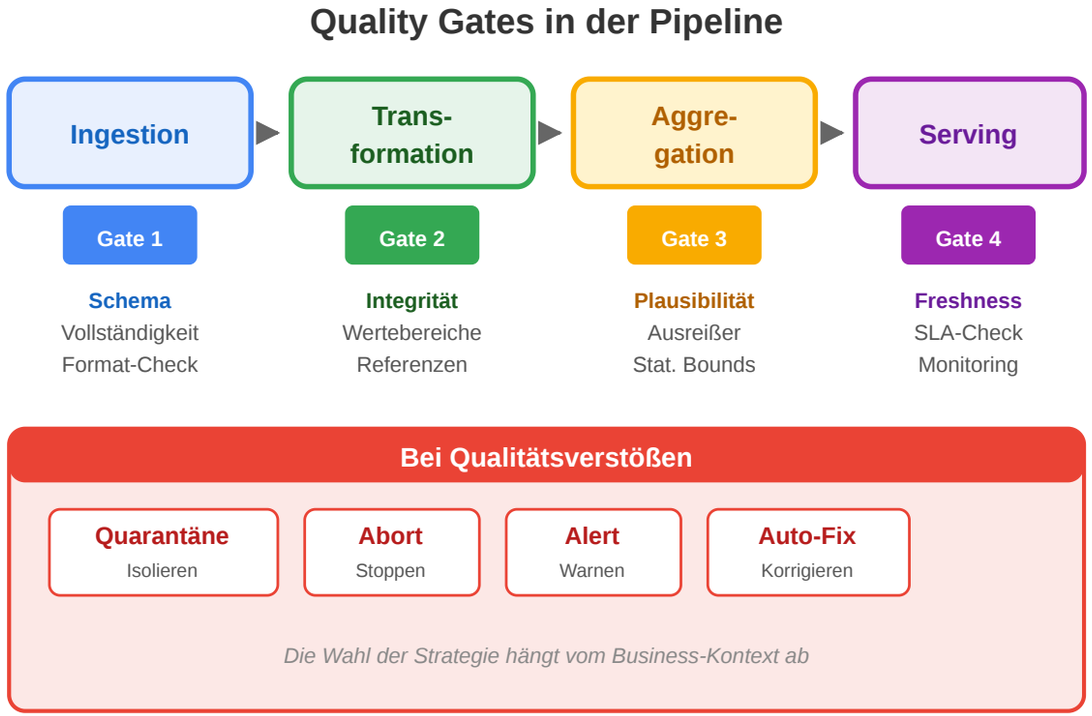
Einsatz: Echtzeit-Dashboards, Trading-Systeme

Managed Cloud-Services

- BigQuery, Redshift, Snowflake
- Serverless, automatisches Scaling
- Pay-per-Query statt Cluster-Management

Datenqualität: Die fünf Dimensionen

Dimension	Prüffrage	Beispiel
Vollständigkeit	Fehlen Werte?	15 % der Bestellungen ohne PLZ
Konsistenz	Widersprechen sich Quellen?	CRM sagt 500 Kunden, DWH sagt 480
Aktualität	Sind Daten frisch genug?	Lagerbestand 6 Stunden veraltet
Genauigkeit	Stimmen die Werte?	GPS-Koordinaten auf falscher Straße
Eindeutigkeit	Gibt es Duplikate?	Gleicher Kunde mit 3 IDs



Qualität ist kein Feature, sondern ein Prozess: kontinuierlich messen, automatisiert prüfen, bei Verstößen alarmieren — wie Software-Tests.

Quality Gates in der Pipeline

Pipeline-Stufe	Quality Gate	Tool-Beispiel
Ingestion	Schema-Validierung, Vollständigkeit, Format	Great Expectations, JSON Schema
Transformation	Referentielle Integrität, Wertebereiche	dbt tests, SQL assertions
Aggregation	Plausibilitätsprüfung, Ausreißer-Erkennung	Statistische Bounds, z-Score
Serving	Freshness-Check, SLA-Monitoring	Airflow SLAs, Monte Carlo

Was passiert bei Qualitätsverstößen?

Strategie	Wann?
Quarantäne	Fehlerhafte Datensätze in separaten Bereich verschieben
Abort	Pipeline stoppen, wenn kritische Schwelle überschritten
Alert	Team benachrichtigen, Daten weiterverarbeiten
Auto-Fix	Imputation fehlender Werte, Deduplication

Keine dieser Strategien ist universell richtig — die Wahl hängt vom Business-Kontext ab: Ein fehlender Sensorwert im IoT-Dashboard ist weniger kritisch als ein fehlerhafter Finanzwert.

Data Governance: Wer ist verantwortlich?

Governance-Frage	Warum wichtig?
Ownership: Wem gehört dieses Datenprodukt?	Ohne Eigentümer wird kein Bug gefixt
Definitionen: Was bedeutet „aktiver Kunde"?	Verschiedene Teams zählen unterschiedlich
Zugriffsrechte: Wer darf was sehen?	DSGVO, PII, Compliance
Lineage: Woher kommen diese Daten?	Debugging, Audit, Vertrauenswürdigkeit
Retention: Wie lange speichern wir?	Speicherkosten, Recht auf Löschung

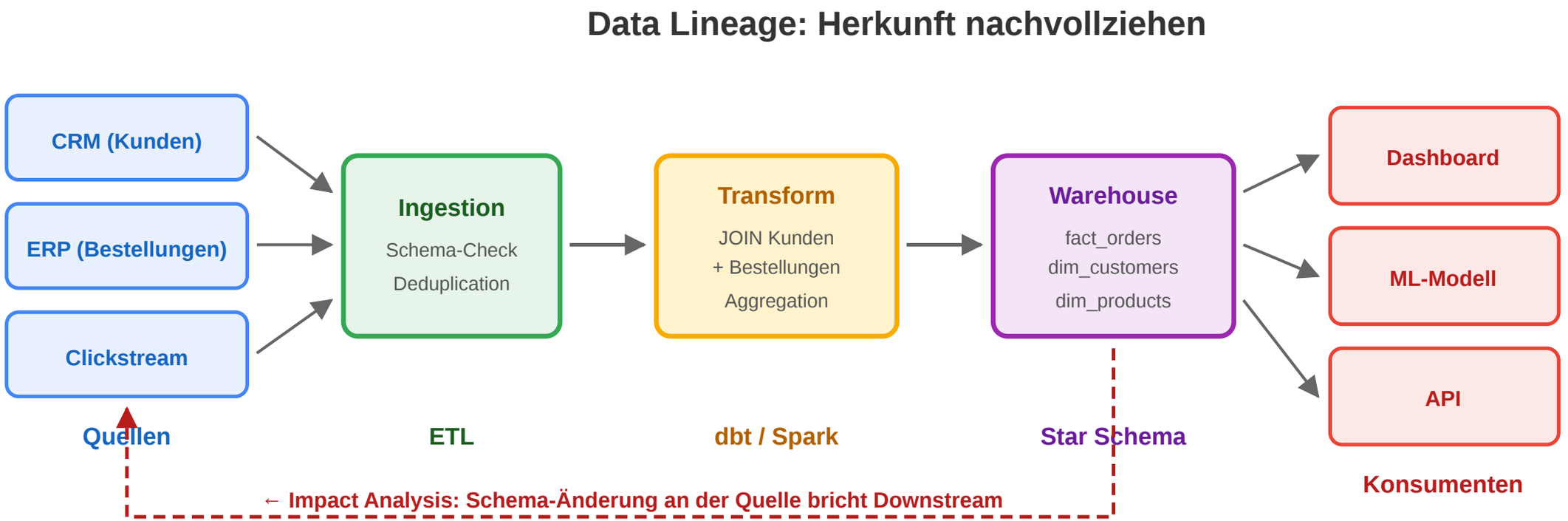
Data Catalog

Ein **Data Catalog** ist das „Google für interne Daten":

- Beschreibung jedes Datensatzes
- Eigentümer und Kontaktperson
- Schema, Datentypen, Beispielwerte
- Lineage-Graph (Herkunft)
- Qualitätsmetriken

Beispiele: DataHub, Apache Atlas, Alation, Google Data Catalog

Data Lineage: Herkunft nachvollziehen



Warum Lineage unverzichtbar ist:

- **Debugging:** Ein Wert im Dashboard ist falsch — welche Transformation hat den Fehler eingeführt?
- **Impact Analysis:** Wir ändern das Schema einer Quelltabelle — welche Downstream-Systeme brechen?
- **Compliance:** Die DSGVO verlangt Auskunft, wo personenbezogene Daten verarbeitet werden
- **Vertrauen:** Data Scientists müssen wissen, ob sie einem Datensatz vertrauen können

Mini-Übung: Quality Check Design

Ein Online-Shop hat eine Produktdaten-Pipeline: Lieferanten-CSV → Ingestion → Transformation → Data Warehouse → Produktkatalog-API.

Entwerfen Sie für jede Pipeline-Stufe **einen konkreten Quality Check:**

Stufe	Was prüfen Sie?	Erwartetes Ergebnis
Ingestion	?	?
Transformation	?	?
Data Warehouse	?	?
API	?	?

Musterlösung

Stufe	Quality Check	Erwartung
Ingestion	CSV hat alle Pflichtfelder (EAN, Name, Preis)	Keine Zeile ohne EAN
Transformation	Preis > 0 und < 100.000 €	Keine negativen oder absurd hohen Preise
Data Warehouse	Anzahl Produkte ändert sich max. 5 % pro Tag	Keine Masseninsertions/-löschungen ohne Grund
API	Response enthält ≥ 95 % der Warehouse-Produkte	Keine stillen Datenverluste beim Serving

Gesamtüberblick: Vom Data Warehouse zu Big Data

Aspekt	Klassisches DWH	Big Data / Cloud
Skalierung	Vertikal (größerer Server)	Horizontal (mehr Nodes)
Datenmodell	Star Schema, strukturiert	Schema-on-Read, polyglot
Verarbeitung	ETL → OLAP	MapReduce, Spark, Streaming
Speicher	RDBMS (Oracle, Teradata)	HDFS, S3, Cloud Storage
Latenz	Stunden (Batch-ETL)	Sekunden (Streaming)
Kosten	Hohe Lizenzkosten, wenige Server	Commodity-Hardware, Pay-per-Use
Qualität	Manuell, selten geprüft	Automatisierte Quality Gates
Governance	Zentral (IT-Abteilung)	Dezentral (Data Mesh / Domain Ownership)

Kein Entweder-Oder: Moderne Datenplattformen kombinieren DWH-Konzepte (Star Schema, SQL) mit Big-Data-Technologien (Spark, Cloud Storage). Das „Lakehouse"-Muster vereint Data Lake und Data Warehouse.

> Data Engineering scheitert selten an Rechenleistung. Es scheitert an unklaren Eigentümerschaften, fehlender Lineage, veralteten Definitionen und ungeprüften Annahmen. Qualität, Governance und

Transparenz sind keine Add-ons — sie sind das Fundament jeder belastbaren Datenplattform.

Wrap-Up

- **Speicherarchitekturen:** Data Warehouse (Schema-on-Write, ACID), Data Lake (Schema-on-Read, günstig, Swamp-Gefahr), Lakehouse (ACID auf Object Storage via Delta/Iceberg, Time Travel, Medallion Architecture).
- **Columnar Formate:** Parquet (Disk) und Arrow (RAM) sind das Standard-Paar für analytische Workloads — columnar komprimiert besser, liest schneller und ermöglicht Predicate Pushdown.
- **MapReduce als Paradigma:** Map (nebeneffektfrei, parallelisierbar) + Reduce (assoziativ, kombinierbar) ist das Grundmuster aller verteilten Datenverarbeitung — von Hadoop (2004) über Spark DataFrames (2015) bis dbt und DuckDB (2020+).
- **Lambda vs. Kappa:** Lambda kombiniert Batch + Stream für exakte und sofortige Ergebnisse — auf Kosten doppelter Codebasis. Kappa vereinfacht auf einen Stream-Pfad — auf Kosten von Replay-Aufwand. Moderne Lakehouses reduzieren den Unterschied.
- **Qualität und Governance:** Technisch funktionierende Pipelines werden ohne Ownership, Lineage und automatisierte Quality Gates fachlich unbrauchbar.

